



Tsinghua University

Modern Computer Architecture Fall 2025

Lecture 06: SS Frontend & Backend

Shuwen Deng

shuwend@tsinghua.edu.cn

2024-Nov.-5

Outline

- Review
 - SuperScalar Pipeline
 - Register Update Unit
- Load-Store Queue
- SuperScalar SpeedUp Measurement
- FrontEnd :
 - Fetch and Decode
 - Branch Prediction
 - RUU/LSQ size
- BackEnd :
 - Result Bus
 - Result Forwarding

Review : SS Pipeline

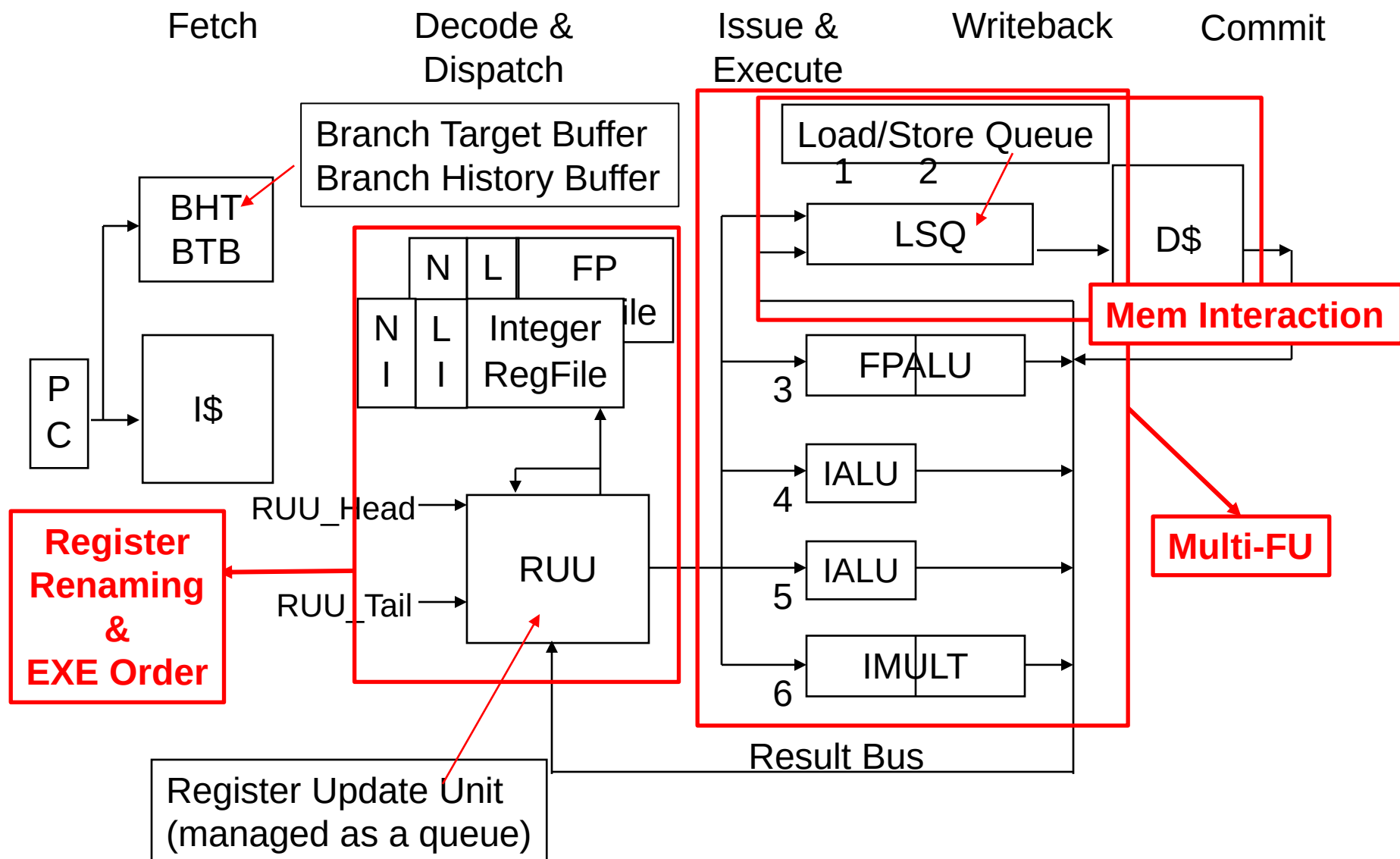


Fig. 9 from Sohi's paper

Review : Register Update Unit (RUU)

- What is RUU?
 - A hardware data structure to resolve data dependencies
 - keeping track of an instruction's data and execution needs
 - commits completed instructions in program order

Review : Register Update Unit (RUU)

- An entry in the RUU

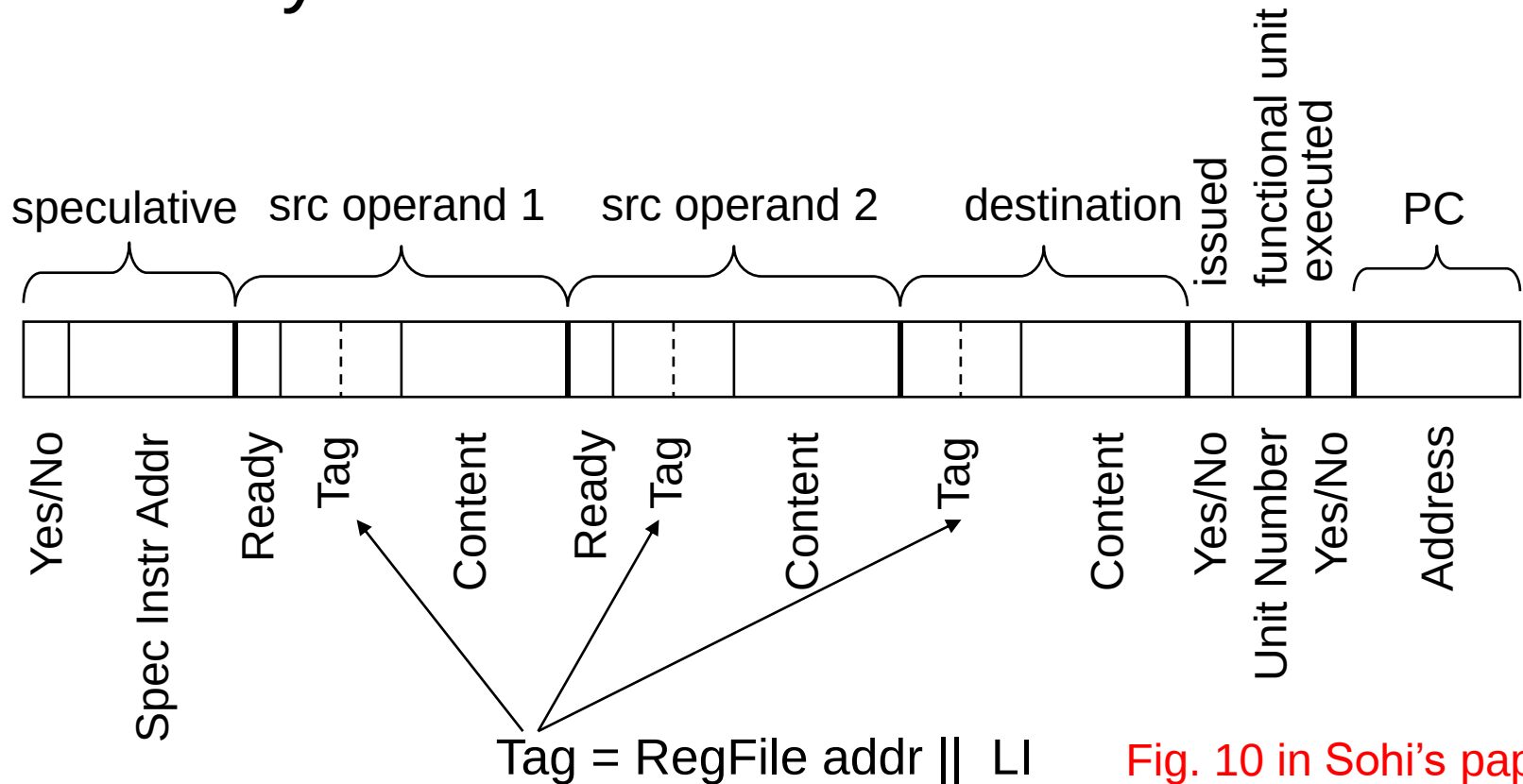
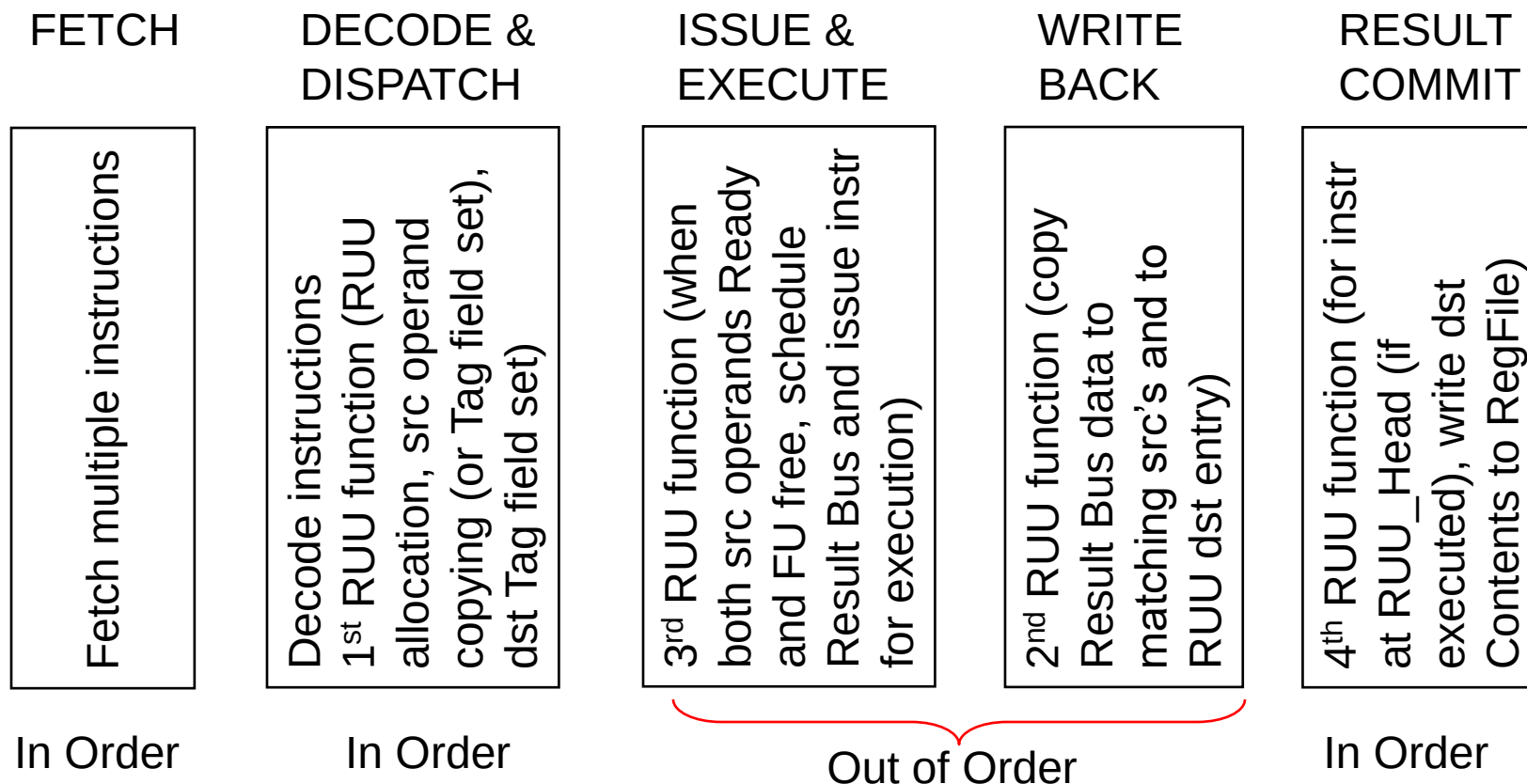


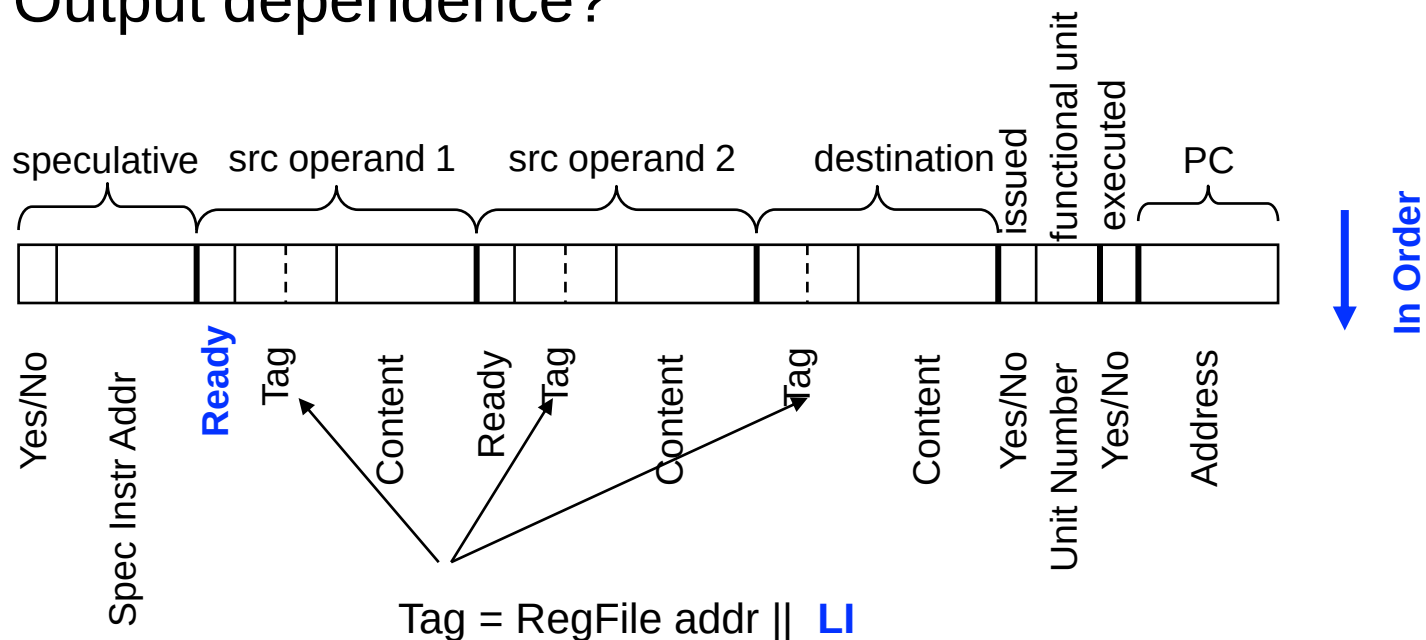
Fig. 10 in Sohi's paper

Review : SS Pipeline



Review: RUU Questions

- **Question to answer after this Part:**
 - In RUU, how to solve the three dependencies ?
 - True data dependence?
 - Anti-dependence?
 - Output dependence?



Register Update Unit (RUU) –Circular Queue

		#	LI			#	LI			#	LI					
RUU_Head	0	1	0	0	3	1	X	X	4	2	1			y	6	n
	1	1	0	0	3	1	3	0	7	1	1			y	4	n
RUU_Tail	2	0	2	1		0	1	1		2	2			n	6	n
	3															

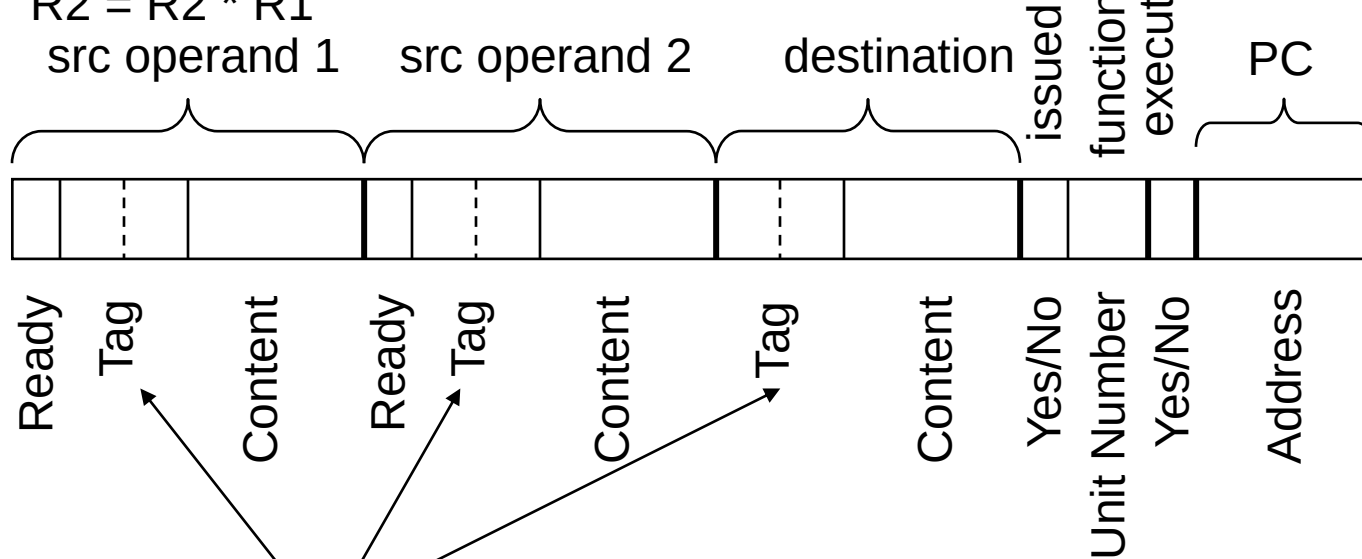
	NI	LI	
\$0	0	0	3
\$1	1	1	?
\$2	2	2	?
\$3	0	0	7

State 0:

Issue: $R2 = R0 * 4$

Issue: $R1 = R0 + R3$

Dispatch: $R2 = R2 * R1$



Register Update Unit (RUU) –Circular Queue

	#	LI		#	LI		#	LI				
RUU_Head	0	1	0	0	3	1	X	X	4	2	1	y 6 n
	1	1	0	0	3	1	3	0	7	1	1	10 y 4 y
RUU_Tail	2	0	2	1		1	1	1	10	2	2	n 6 n
	3											

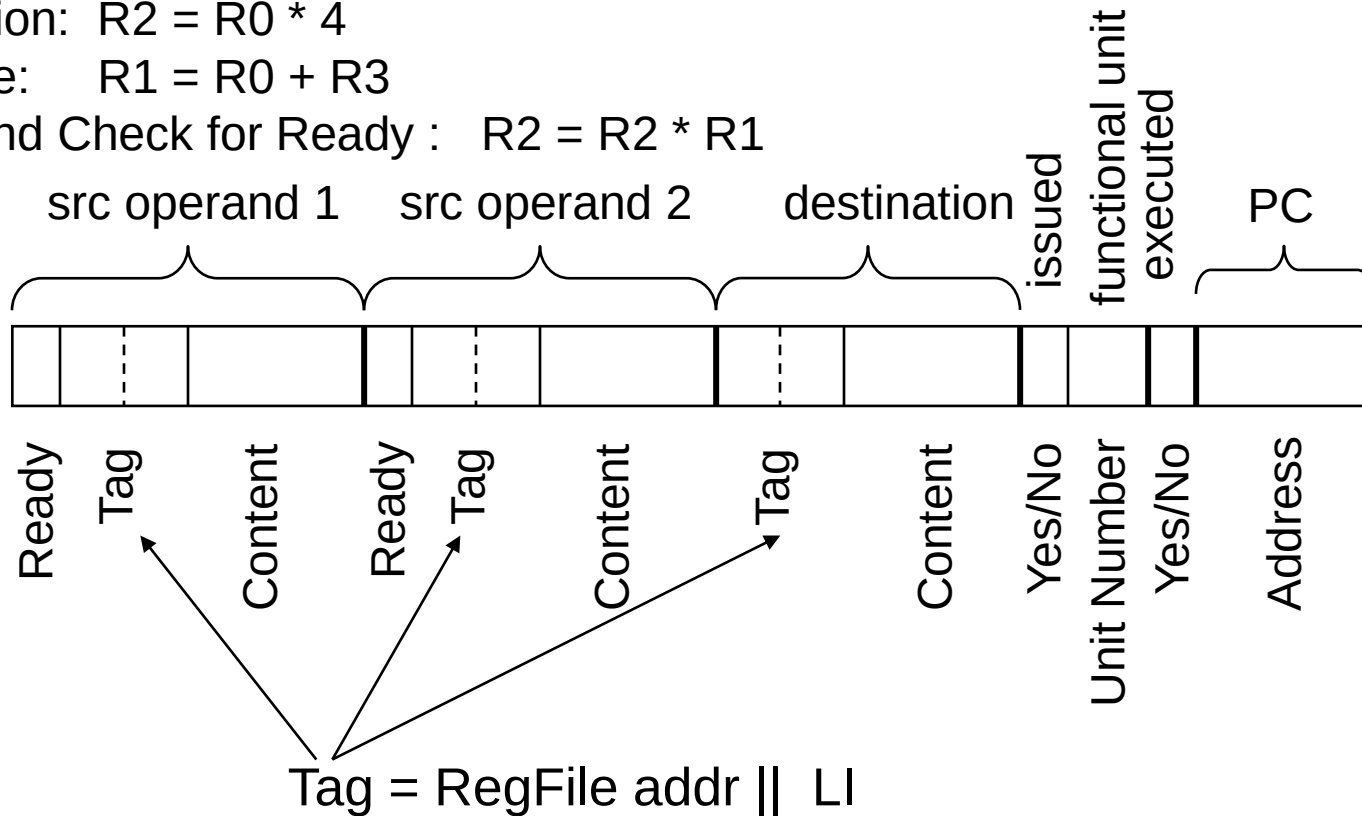
	NI	LI	
\$0	0	0	3
\$1	1	1	?
\$2	2	2	?
\$3	0	0	7

State 1:

In Execution: $R2 = R0 * 4$

Finish Exe: $R1 = R0 + R3$

In RUU and Check for Ready : $R2 = R2 * R1$



Register Update Unit (RUU) –Data Dependence

Monitors the Result Bus to resolve **true dependencies** and to do write back of result data to the RUU (**Ready bit**)

Solves **anti-dependencies** through LI (making sure that the source fields get updated only for **the correct version of the data**)

Solves **output-dependencies** by **program order** in RUU

Suppose:

In Execution: $R2 = R0 * 4$

$R2 = R2 * R1$

Finish Exe: $R1 = R0 + R3$

In RUU and Check for Ready :

		#LI				#LI				#LI					
RUU_Head	0	1	0	0	3	1	X	X	4	2	1		y	6	n
	1	1	2	1		1	1	0	4	2	2		y	4	n
RUU_Tail	2	1	0	0	3	1	3	1	7	1	1		y	5	y
	3														

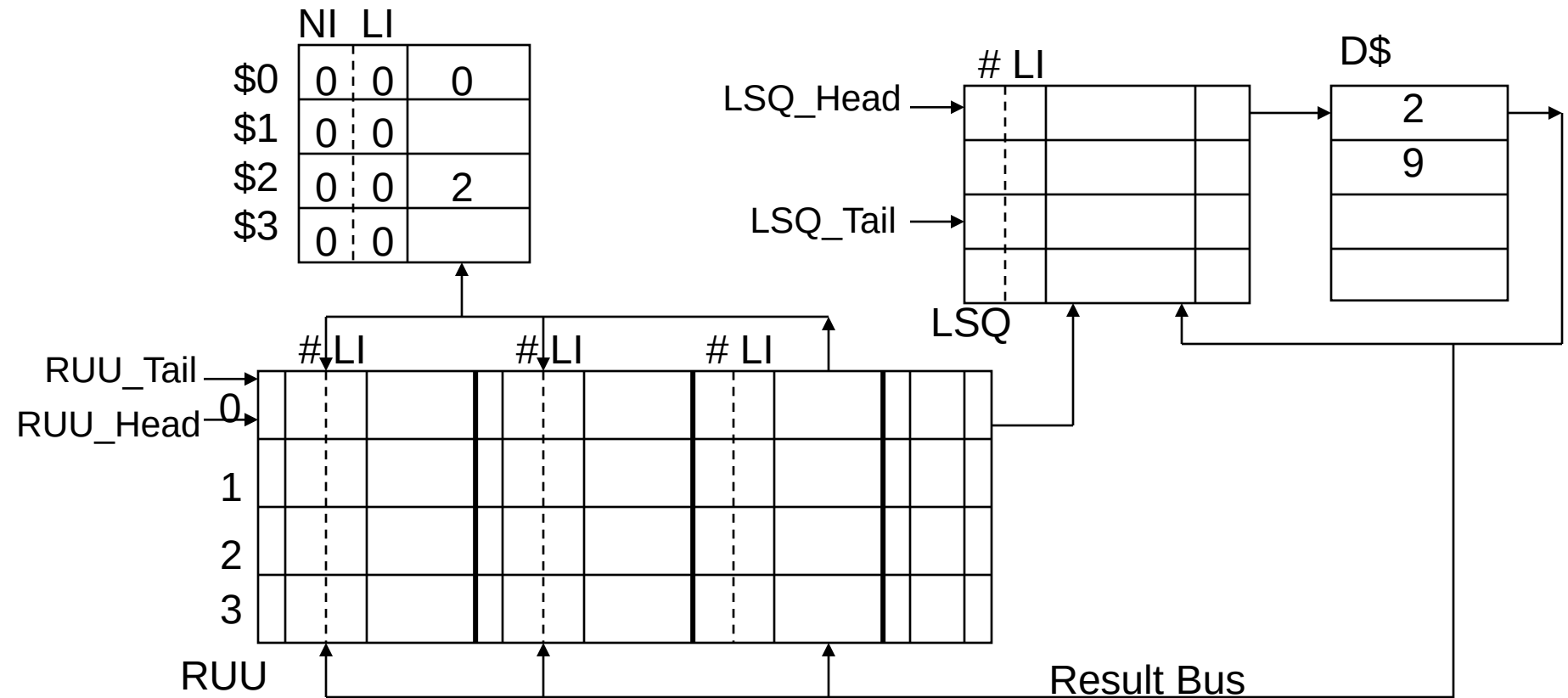
MicroOperations of Load and Store

- Loads and Stores in RUU as two (micro)instr's – one to compute the addr and one to do the memory operation
 - Load lw R1, 2(R2) becomes addi R0,R2, 2
 lw R1,R0
 - Store sw R1, 6(R2) becomes addi R0,R2, 6
 sw R1,R0
- Key Points
 - Each LSQ entry consists of a Tag and a Content field.
 - The LI counter allows for multiple instances of stores (writes) to a memory address
 - Release LSQ entry when a load completes (the D\$ returns the data on the Result Bus) or a store commits (in program order)
 - Instruction dispatch is blocked if there is not a free LSQ entry and two free RUU entries

Load & Store RUU & LSQ Operation

Load
addi R0,R2,2
lw R1,R0

Store
addi R0,R2,6
sw R1,R0



Load & Store RUU & LSQ Operation

Load

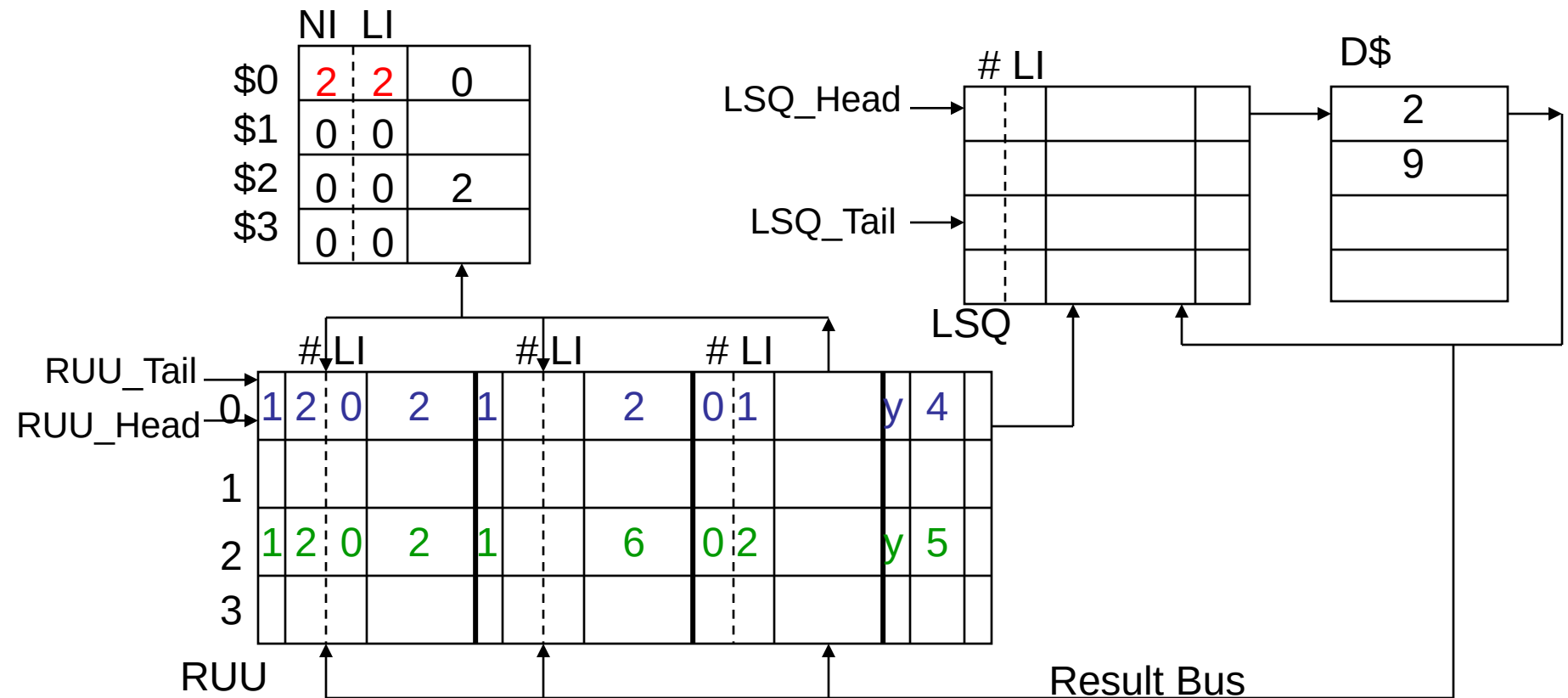
addi R0,R2,2

lw R1,R0

Store

addi R0,R2,6

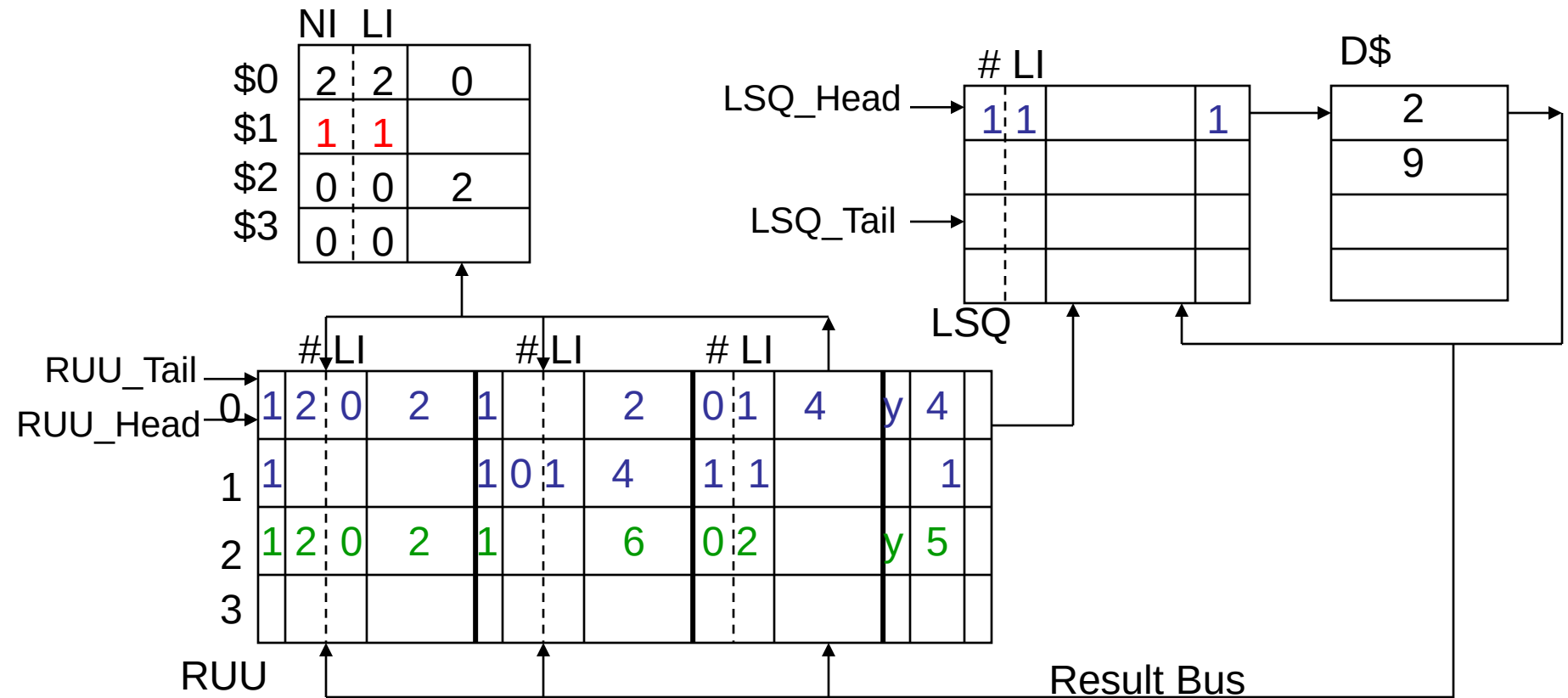
sw R1,R0



Load & Store RUU & LSQ Operation

Load
addi R0,R2,2
lw R1,R0

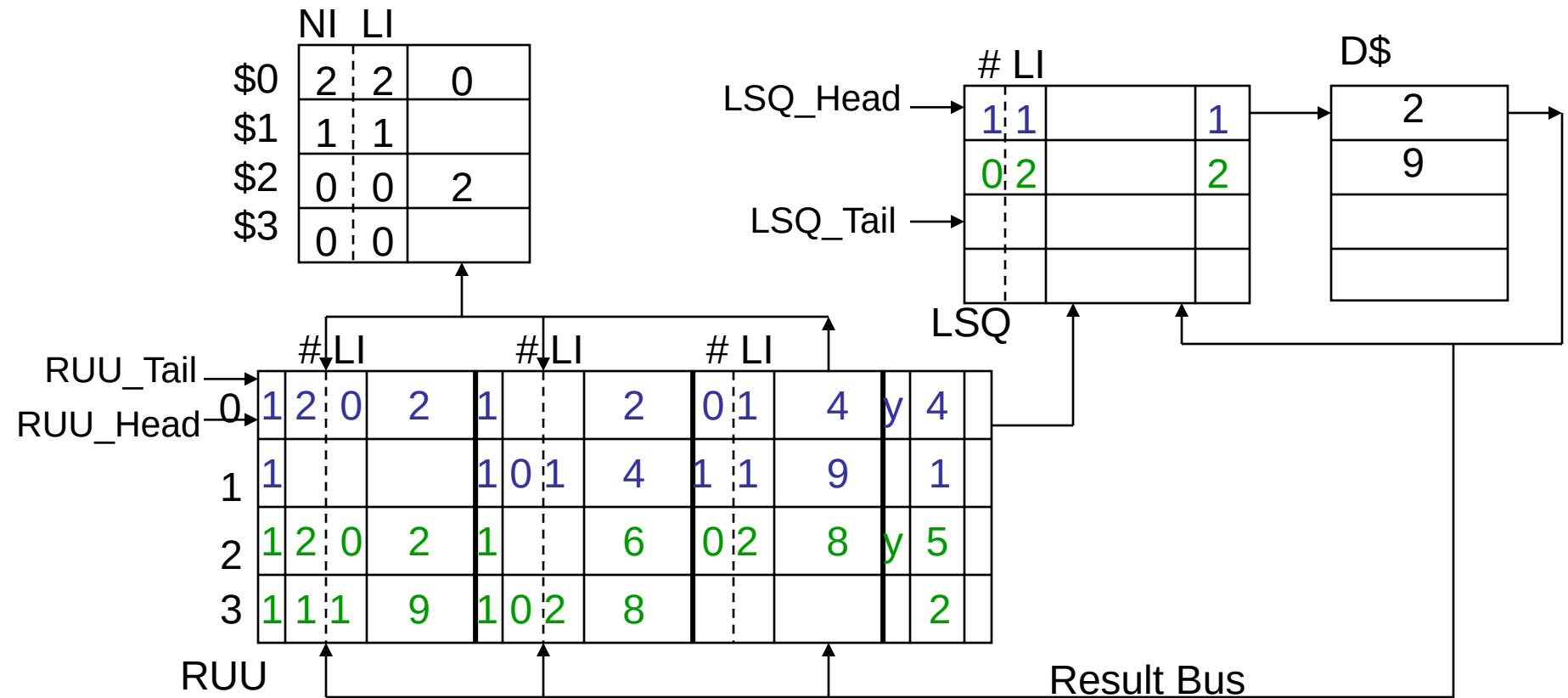
Store
addi R0,R2,6
sw R1,R0



Load & Store RUU & LSQ Operation

Load
addi R0,R2,2
lw R1,R0

Store
addi R0,R2,6
sw R1,R0



Memory Location Data Dependencies

- True, anti- and output dependencies in memory as well
 - Memory storage conflicts are infrequent since memory locations are not reused in the same way as registers
 - The number of **true dependencies is small** (loads and stores are less than 30% of the instruction mix)
- **Load bypassing**
 - Loads are allowed to bypass previous stores
 - Check for load data dependency on previous uncommitted stores
 - Facilitated by a common load and store queue
- **No antidependencies**
 - Stores are not allowed to bypass previous loads
- **No output dependencies**
 - Stores are committed to the D\$ in program order

Loads from Memory

- Compare in LSQ, when a load's address is known
 - **Match load** in the LSQ, current load does not need to be issued
 - **Match store** in the LSQ, stop and matching pending store can directly supply the destination Content for the current load
 - **No match**, the load is issued to the LSQ and executed
- When the RUU# of the load instr appears on the Result Bus (along with the memory data)
 - Completes by **updating the RUU**
 - **Releasing the LSQ entry**
 - RUU entry is released on **load commit**

Stores to Memory

- Compare in LSQ, when a store's address is known
 - **Match load** in the LSQ, the current store is issued to the LSQ
 - **Match store** in the LSQ, the current store is issued with LI+1
 - **No match**, the store is dispatched to the LSQ
- **Stores are held in the LSQ until ready to commit**
 - Its partner instr reaches the RUU_Head
 - The data and address are sent to the D\$
 - the RUU and LSQ entries are released

Summary of Last Week

- ILP
 - (Super)Pipeline
 - Multi-Issue : VLIW (static) / SS(dynamic)
- Strategies in RUU to Avoid Hazard
 - Ready Bit: True data dependence
 - Latest Instance : Anti-dependence (RAW)
 - In-Program-Order Commit : Output dependence
- Interaction with Memory : LSQ

Outline

- Review
 - SuperScalar Pipeline
 - Register Update Unit
- Load-Store Queue
- SuperScalar SpeedUp Measurement
- FrontEnd : Branch Prediction
- BackEnd : Result Bus
- Performance Evaluation

Speedup Measurements

- The speedup of the SS processor is

$$\text{speedup} = s_n = \frac{\# \text{ scalar cycles}}{\# \text{ superscalar cycles}}$$

- Assumes scalar and superscalar machines have the same IC(instruction count) & CR(clock rate)
- To compute average speedup performance

Arithmetic mean

$$AM = \frac{\sum_{i=1}^n s_i}{n}$$

Harmonic mean assigns a larger weighting to the programs with the smallest speedup

$$HM = n / \left(\sum_{i=1}^n 1/s_i \right)$$

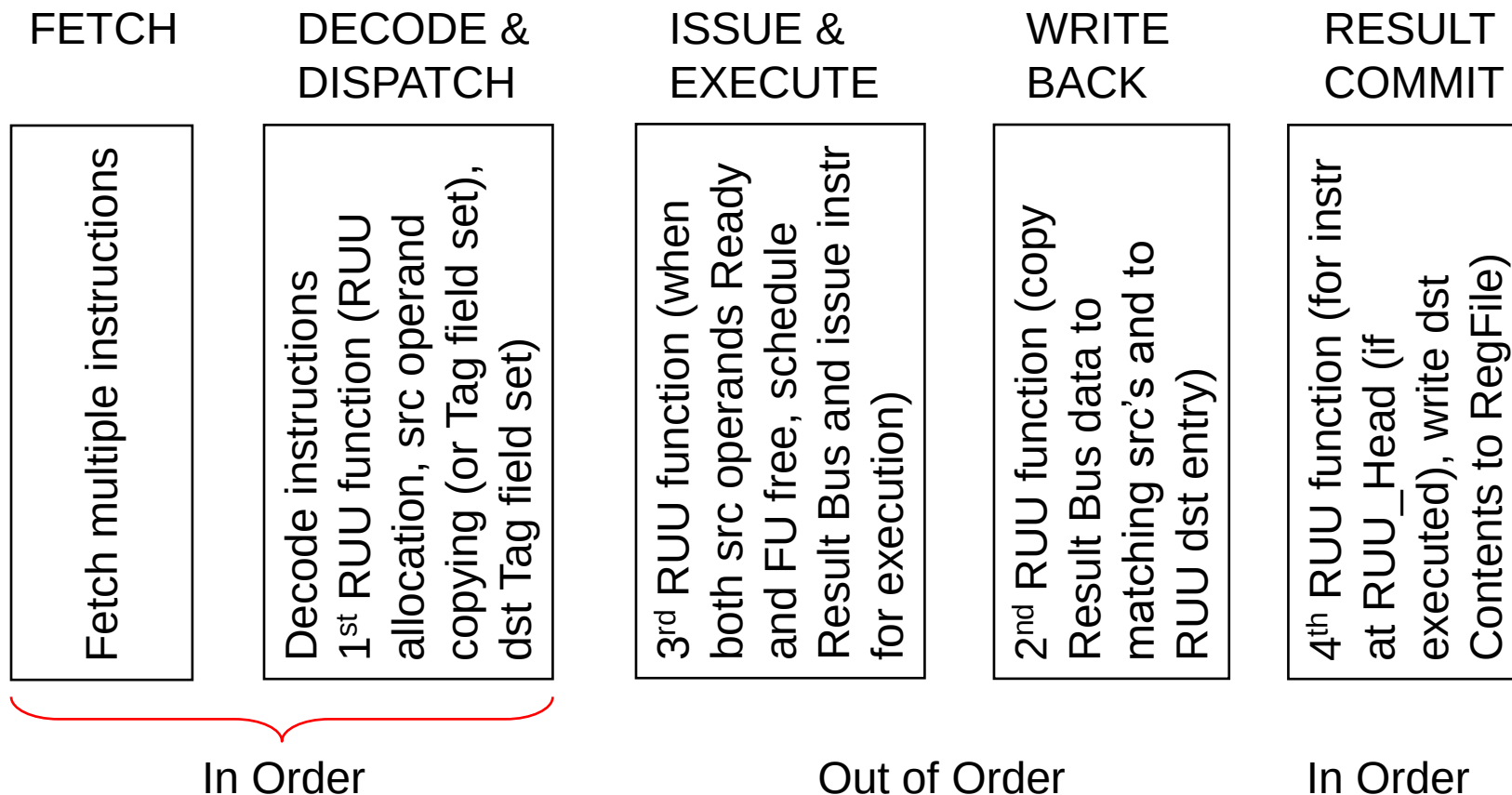
Speedup Measurements

EX : Two programs with same scalar cycles, with a SS speedup of 2 for program1 and 25 for program2.

- $AM = \frac{1}{2} \times (2 + 25) = 13.5$

- $HM = \frac{2}{\frac{1}{2} + \frac{1}{25}} = \frac{2}{0.54} = 3.7$

SS Pipeline



SS Fetch and Decode Pipeline Stages

FETCH

Fetch 4 instructions
Align and merge
Access BHT/BTB for PC,
PC+4, PC+8, PC+12

DECODE & DISPATCH

Decode 4 instructions
1st RUU function (RUU
allocation, src operand
copying (or Tag field set), dst
Tag field set)
If branch, use BHT info to
determine next PC value and
whether the I\$ or BTB instr's
are the next fetch set

ISSUE & EXECUTE

3rd RUU function (when both
src operands Ready and FU
free, schedule Result Bus
and issue instr for execution)

In Order

Out of Order

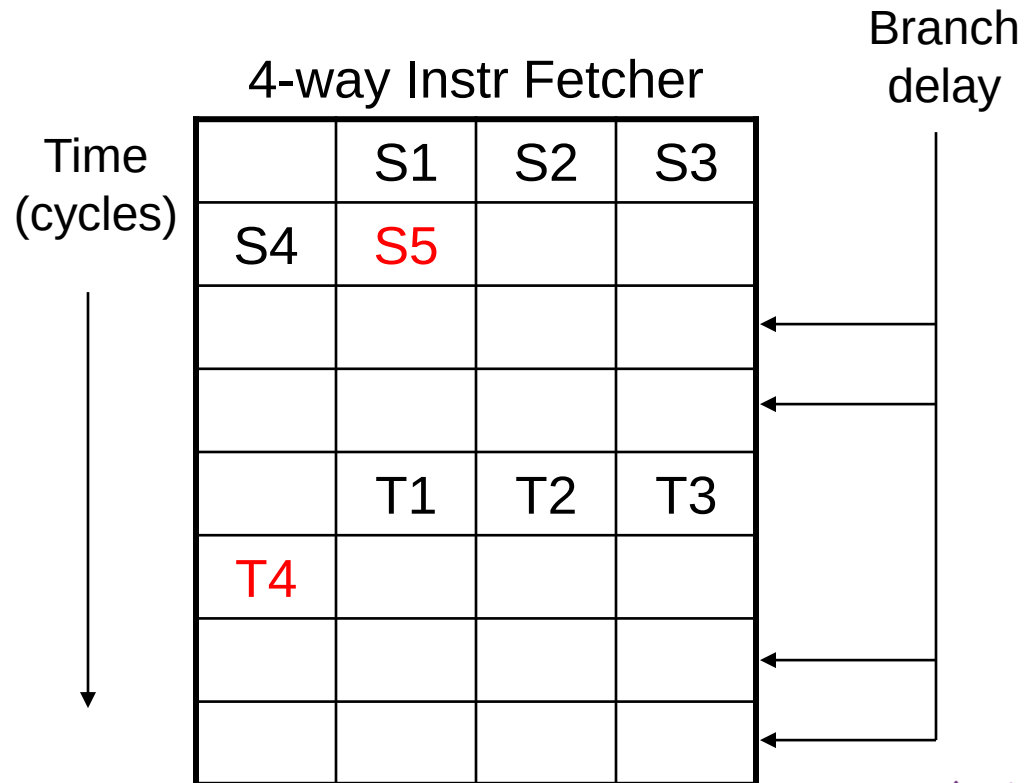
Instruction Fetch Sequences

- Instruction run – number of instructions (run length) fetched between **taken** branches
 - Instruction fetcher operates most efficiently when processing long runs – unfortunately runs are usually quite short

❑ The **average** run length is about **six** instructions

❑ Instruction bandwidth of only 1.125 instructions per cycle

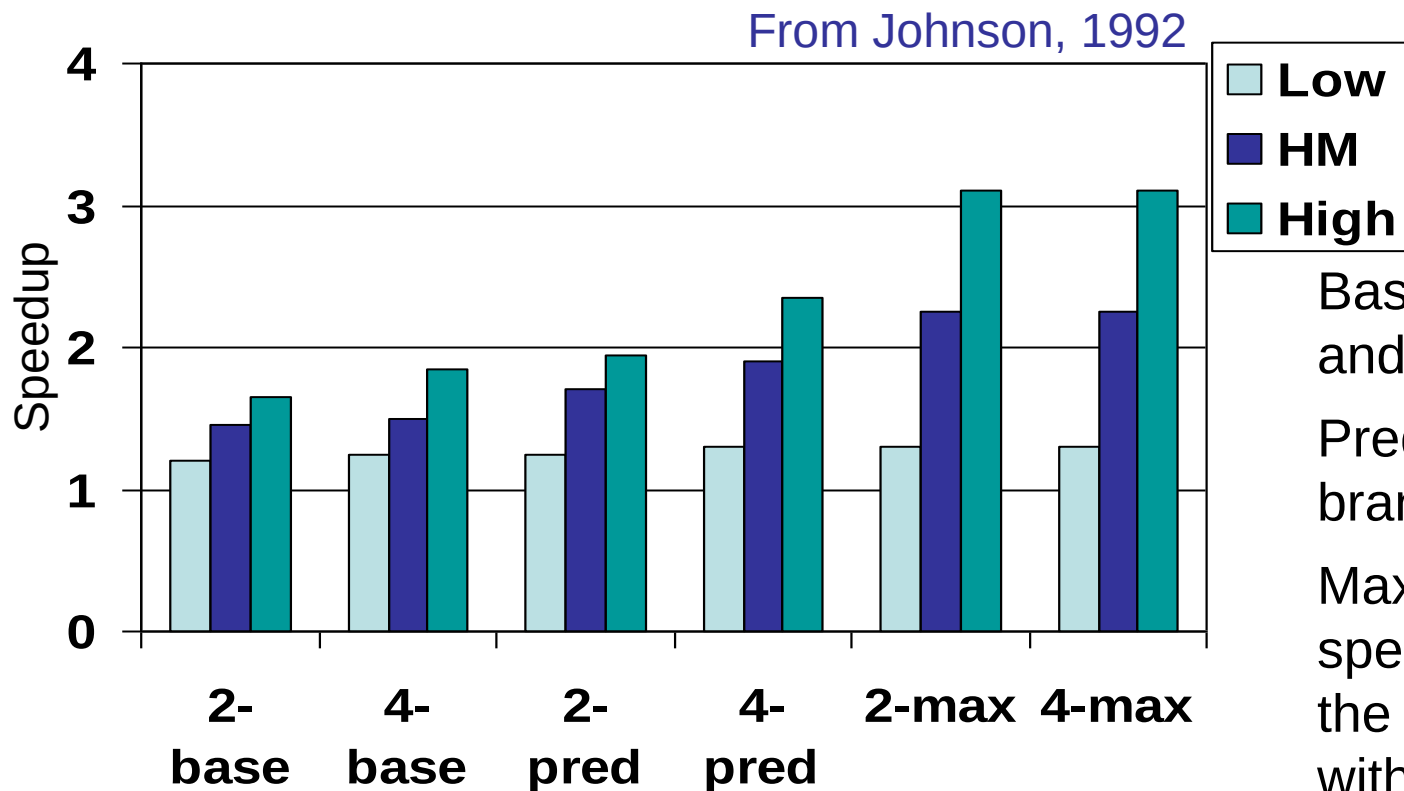
9 instructions in 8 cycles



Instruction Fetch Inefficiencies

- Fetcher can't provide adequate bandwidth to the decoder to exploit the available ILP because
 - Decoder is idle while the outcome of the branch is determined
 - Can (mostly) fix with dynamic branch prediction
 - Instruction fetch misalignment prevents the decoder from operating at full capacity even when the decoder *is* processing valid instructions
 - The fetcher can **align** fetched instructions to avoid wasted decoder slots
 - If supported by dynamic branch prediction, the fetcher can also **merge** instructions from different runs
- Aligning and merging can only be done if the fetcher has the sufficient bandwidth (i.e., the fetch rate is faster than the decode rate)

Speedups of Fetch Alternatives



Base: no prediction and no alignment

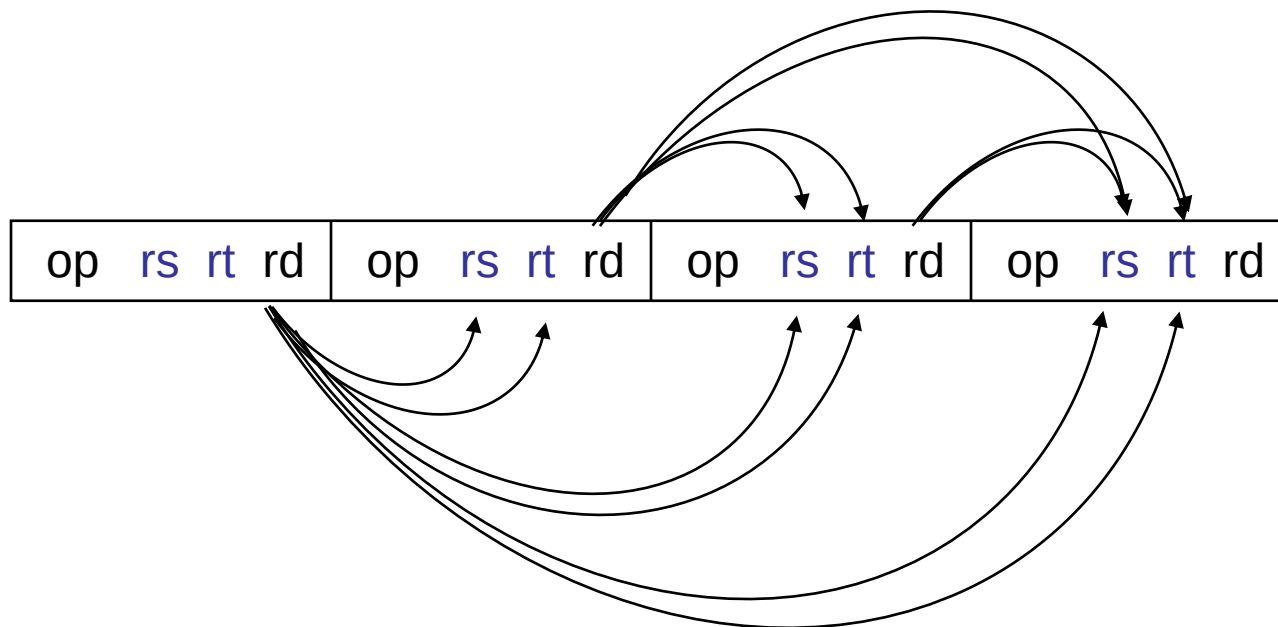
Pred: dynamic branch prediction

Max: the max speedup allowed by the EXE hardware, with no Fetch limitations

- A 4-way instr fetcher outperforms a 2-way instr fetcher
 - It has twice the potential instruction bandwidth
 - But it requires twice as much decoder hardware to keep up (e.g., in decoders and in ports and buses)

4-Way Decoder Implementation

- A 4-way instr fetcher has higher fetch bandwidth but at what cost ?
 - 12 dependency checks between the 4 decode instruction



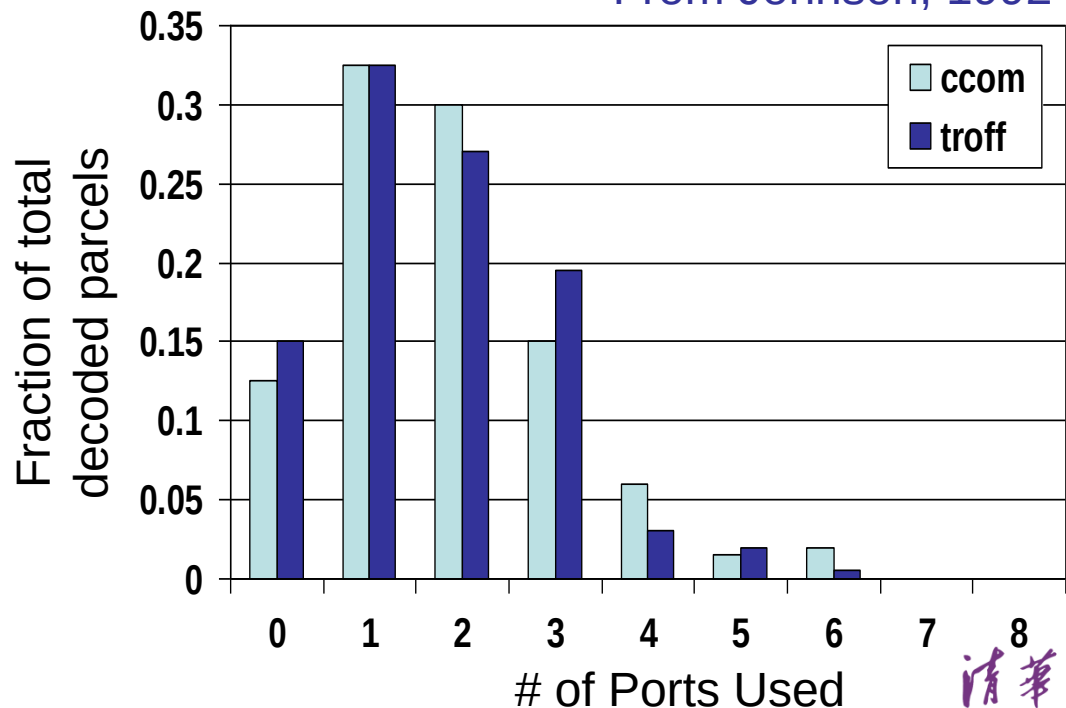
- 8 read ports on the RegFile and 8 write ports to the RUU and 8 buses to distribute those **source operands** (or their RegFile addr || LI)

Reducing 4-Way Decoder Hardware

- Limiting the number of RegFile read ports, buses and RUU write ports is acceptable since
 - Not all decoded instructions access two registers
 - Not all decoded instruction are valid (because of misalignment)
 - Some decoded instructions have dependences on one or more simultaneously decoded instructions

- ❑ The register demand means a 8 port capacity would be wasted
- ❑ 4 ports reduces average performance < 2%

From Johnson, 1992



Arbitrating Read Port and Bus Usage

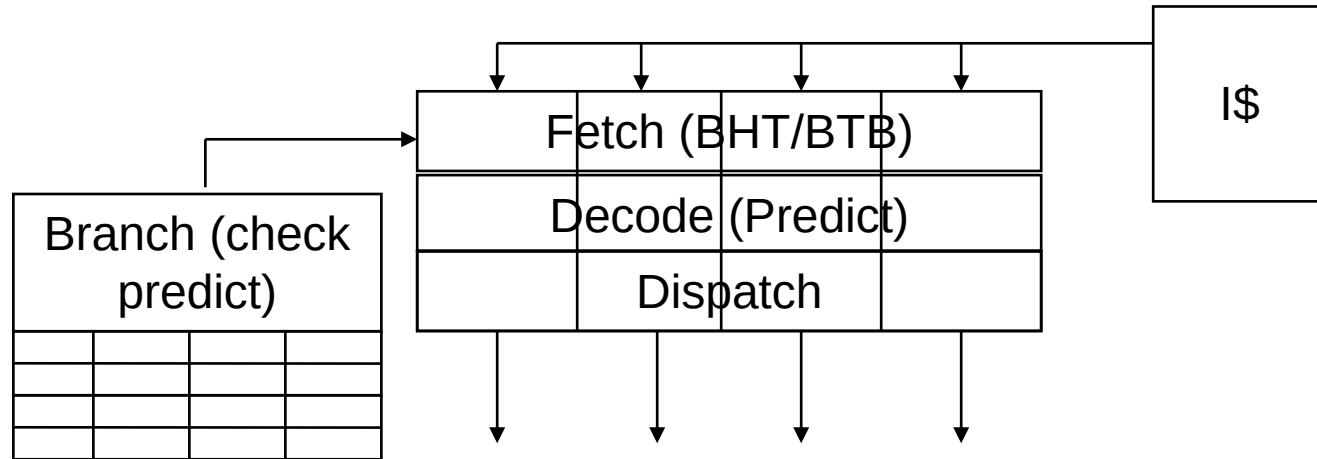
- If only 4 read ports and 4 buses are provided, have to determine which instr's get first access to them
 - If more than 4 ports are needed to dispatch the decoded instructions, then instruction fetch and decode must stall
- Prioritized register identifier selection for port usage must be accomplished within about **half a processor cycle**
 - If the first decoder position instr (i.e., the first instr in program order) requires register access, it is always enabled on the first and second, if it has two source operands, ports
 - Such **arbitration** continues in sequence for the second, third, and fourth decoder position instr's

SS Branch Prediction

- In a scalar pipeline we needed
 - A mechanism to predict the branch outcome: a **BHT** (branch history table) in the fetch stage
 - A way to fetch two instructions – the sequential instruction (I\$) *and* the branch target instruction (**BTB** (branch target buffer))
 - A way to ensure that instructions active in the pipeline following the branch didn't change the machine state until the branch outcome was known
 - Allowed to complete (in order commit) on correct prediction
 - Flushed on mispredict and restart
- With a SS machine, it is possible to have many such instructions after predicted branches *active* in the pipeline
 - Flag instructions following branches as **speculative** until the branch outcome is known

Implementing Branches

- A SS processor could have more than one branch per fetch set and could have several uncompleted branches

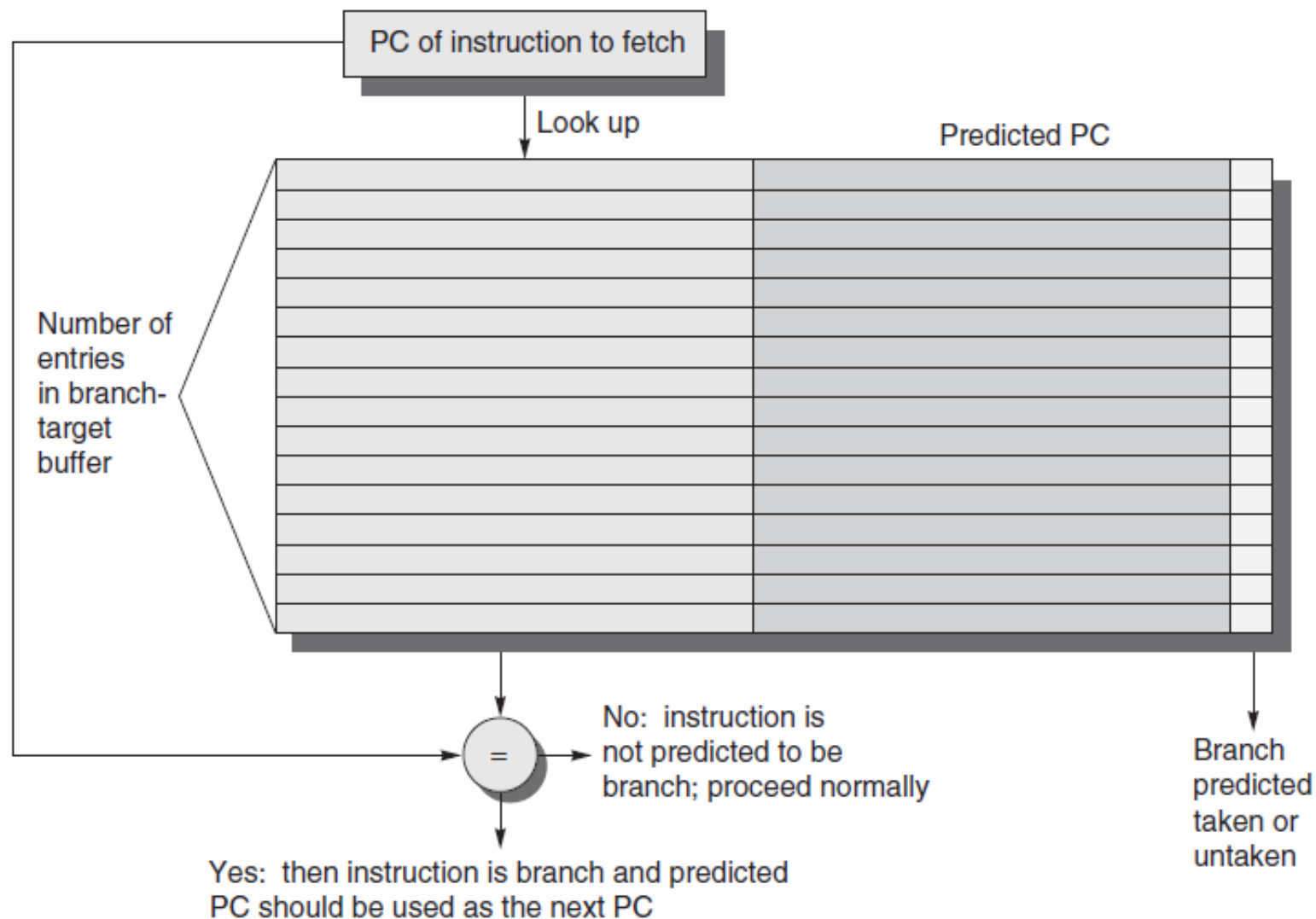


- ❑ Must access BHT/BTB for all branch instr's in the fetch set during fetch to reduced branch delay (i.e., need a 4 read-port BHT/BTB for 4-instr fetcher)

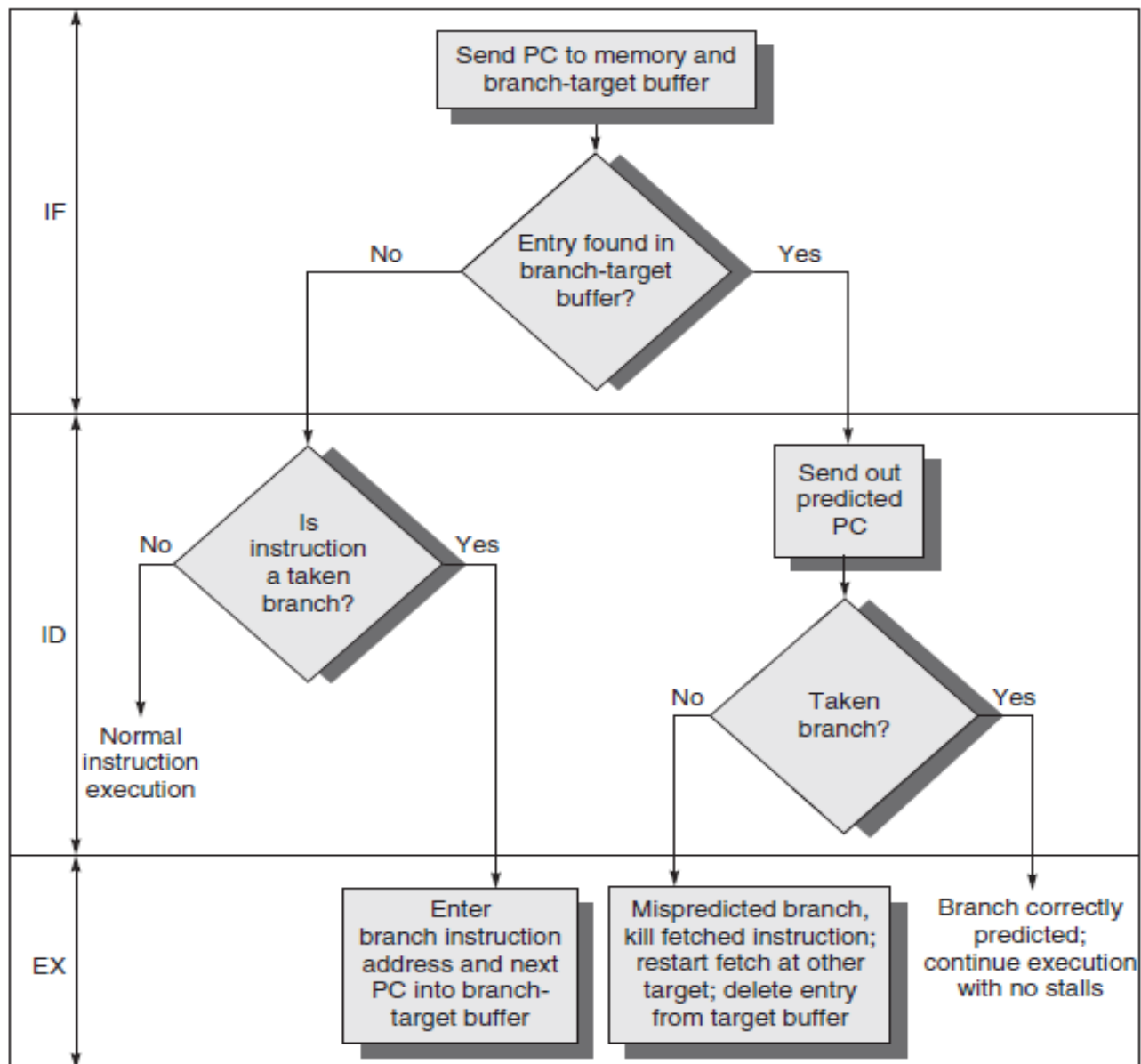
Pass BHT information to decode stage

After decode, choose between I\$ set and BTB sets to determine the next fetch set

Fetch: BTB (Branch Target Buffer)

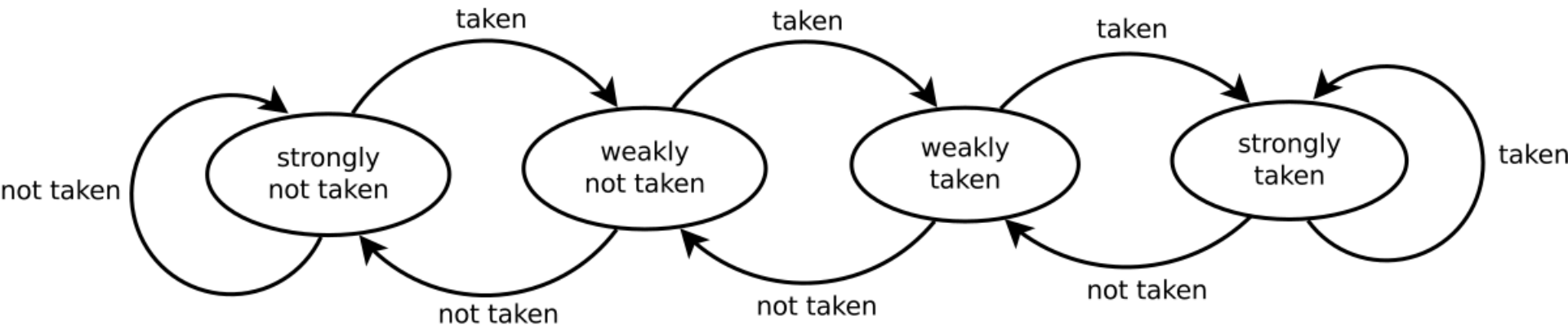


Handling an instruction with a BTB



Decode: BHT (Branch History Table)

- Local Branch Predictor
 - keeps a separate history record for each conditional jump.
 - Example : Standard 2-bit Predictor



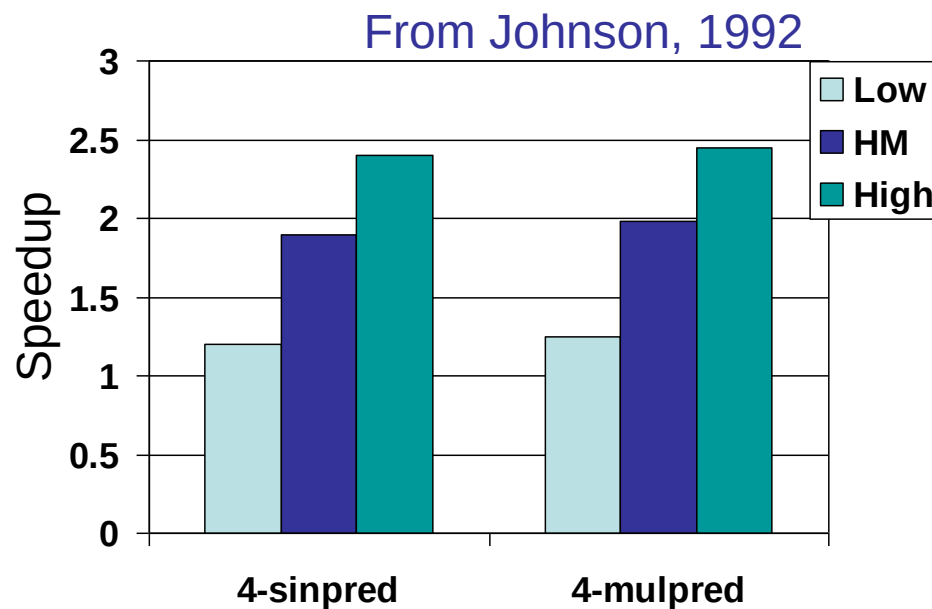
Decode: BHT (Branch History Table)

- Global Branch Predictor:
 - does not keep a separate history record for each conditional jump.
 - Example : Two-level adaptive predictor,
- Combine Local & Global Branch Predictor
 - Tournament predictor

Decoding & Dispatching Branches

- While multiple branches could be dispatched per cycle, incur only a slight performance decrease (about 2%) from imposing a decoder limit of one branch per fetch set, since typically only one branch per cycle can be executed (usually only have one branch FU)

□ Having minimum branch delay is more important than decoding multiple branches per cycle.

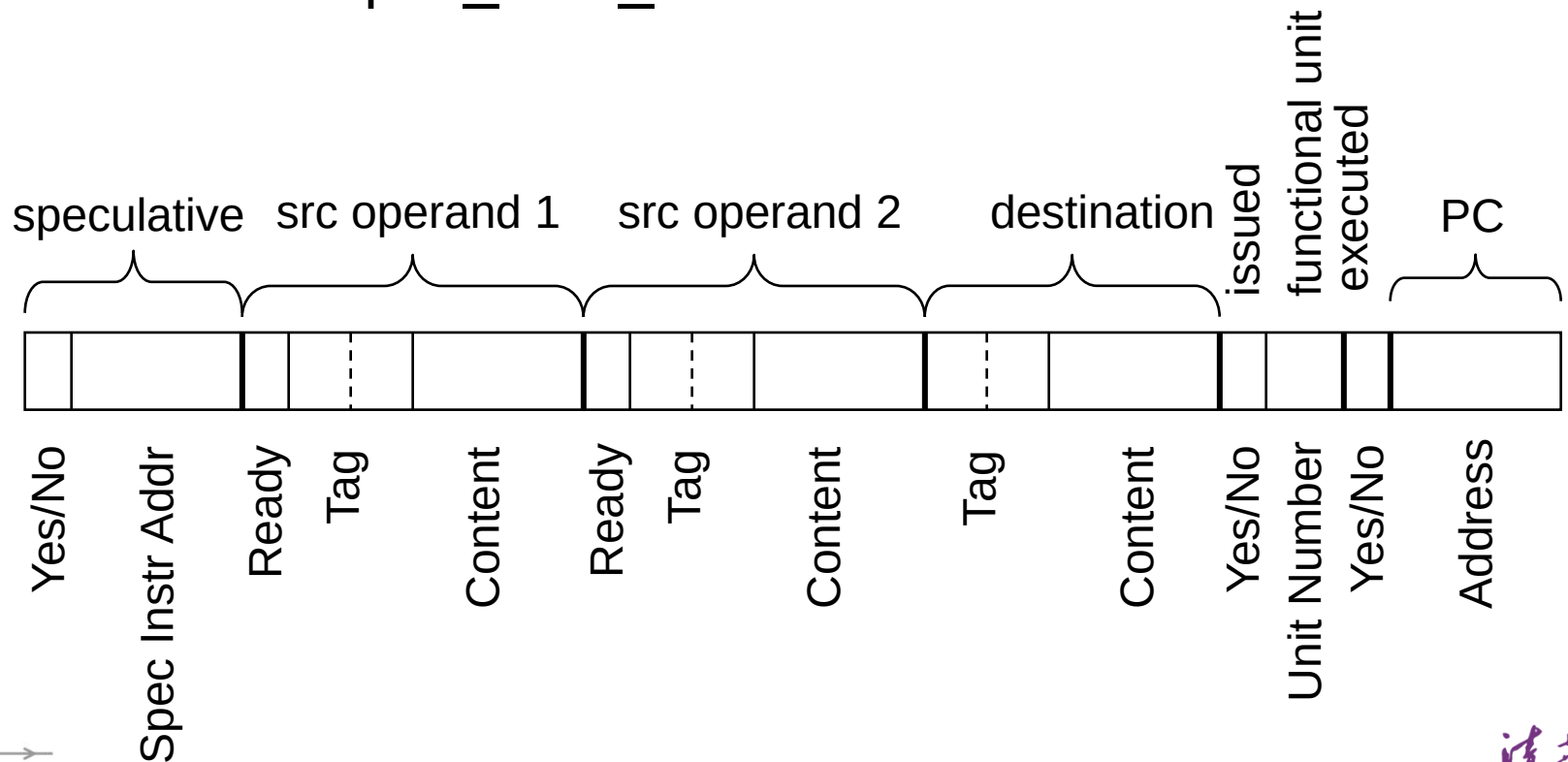


Speculative Instructions

- Speculation – The processor (or compiler) guesses the outcome of an instruction (e.g., branches, loads) so as to enable execution of other dependent instructions
 - One of the most important methods for finding more ILP in SS and VLIW processors
- Producing correct results requires result checking, recovery and restart **hardware** mechanisms
 - **Checking mechanisms** to see if the prediction was correct
 - **Recovery mechanisms** to cancel the effects of instructions that were issued under false assumptions
 - **Restart mechanisms** to reestablish the correct sequence
 - For branches the correct PC (program counter) restart value is known when the branch outcome is determined

RUU Speculation Field Support

- For dependent speculative instr's, **the speculative flag** is set to Yes until the outcome of the driving instr (i.e. the branch) is determined.
- An associate comparison** of that branch's PC addr and the RUU's Spec_Instr_Addr fields can be done.



Branch Execution

- If the branch was **not** mispredicted, then the branch and its trailing instructions can commit when at RUU_Head
- If the branch **was** mispredicted, then **all subsequent instr's must be discarded** (even though subsequent *branches* may have been correctly predicted)
- When there is an exception, **all of the RUU entries are discarded** in a *single cycle* and instruction stream fetching restarts on the next cycle.
- The RUU provides an easy way to discard instructions coming after a mispredicted branch.

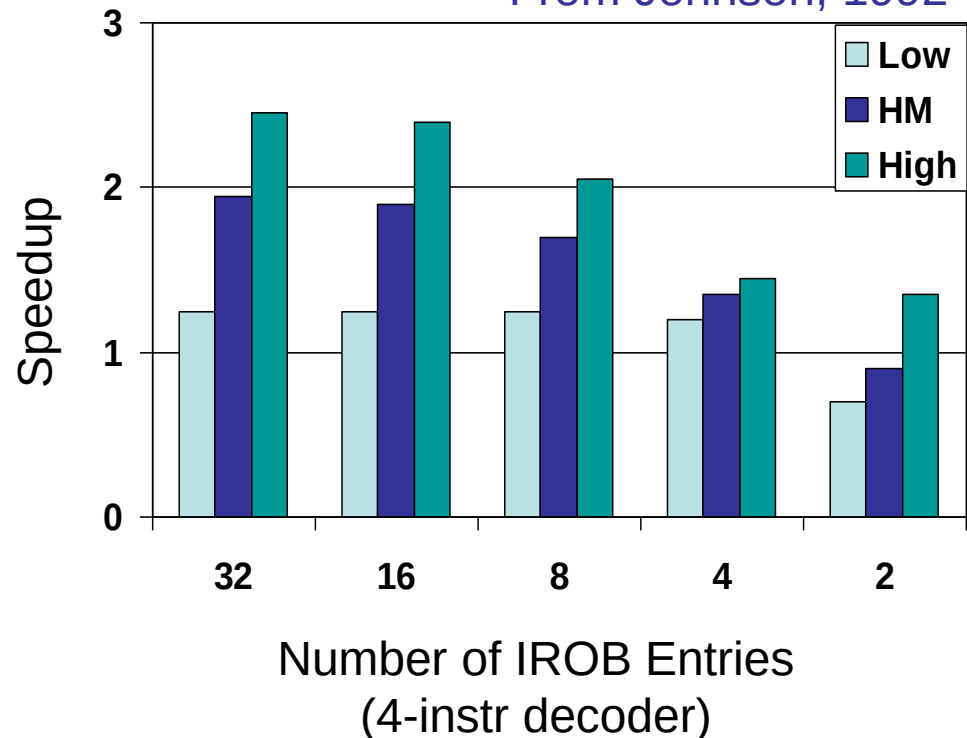
Effects of RUU Size on Performance

- Since instruction decoding must stall when there is no free RUU entry, the RUU should be large enough to accept *all* instructions during the expected dispatch-to-commit time period

- ❑ Performance decreases markedly with 8 and 4 entries
- ❑ For 2 entries the performance is worse than the scalar processor

Why?

From Johnson, 1992



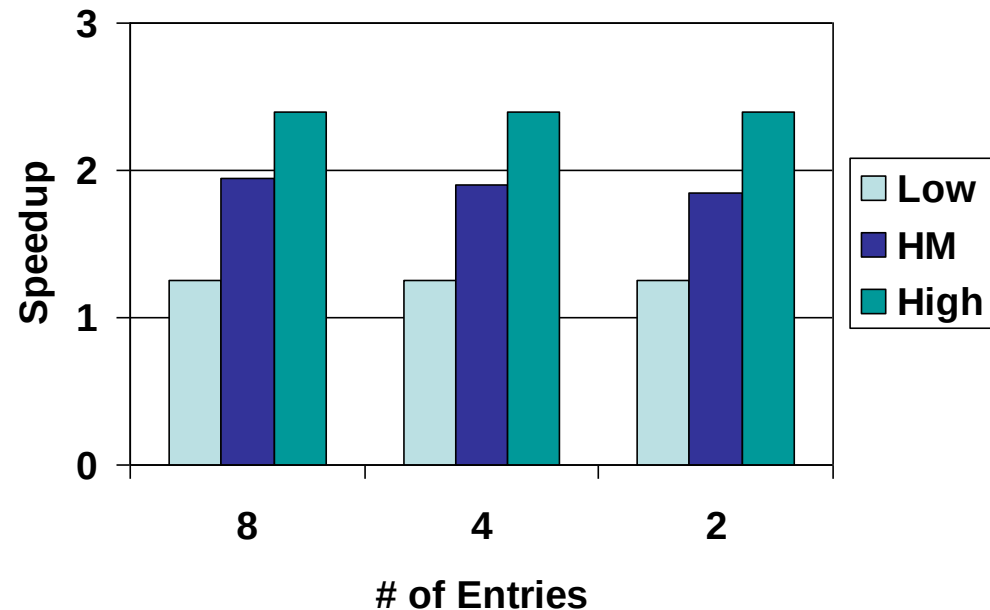
Effects of LSQ Size on Performance

- Instruction decoding must stall when there is no free LSQ entry, and LSQ should be large enough but the LSQ size has relatively little impact on performance

❑ With a 4-way decoder, a 4-entry LSQ incurs a 1% speedup loss over an 8-entry LSQ

❑ Smaller LSQ facilitate dependency checking

From Johnson, 1992



SS Fetch and Decode Pipeline Stages

FETCH

Fetch 4 instructions
Align and merge
Access BHT/BTB for PC,
PC+4, PC+8, PC+12

In Order

DECODE & DISPATCH

Decode 4 instructions
1st RUU function (RUU
allocation, src operand
copying (or Tag field set), dst
Tag field set)
If branch, use BHT info to
determine next PC value and
whether the I\$ or BTB instr's
are the next fetch set

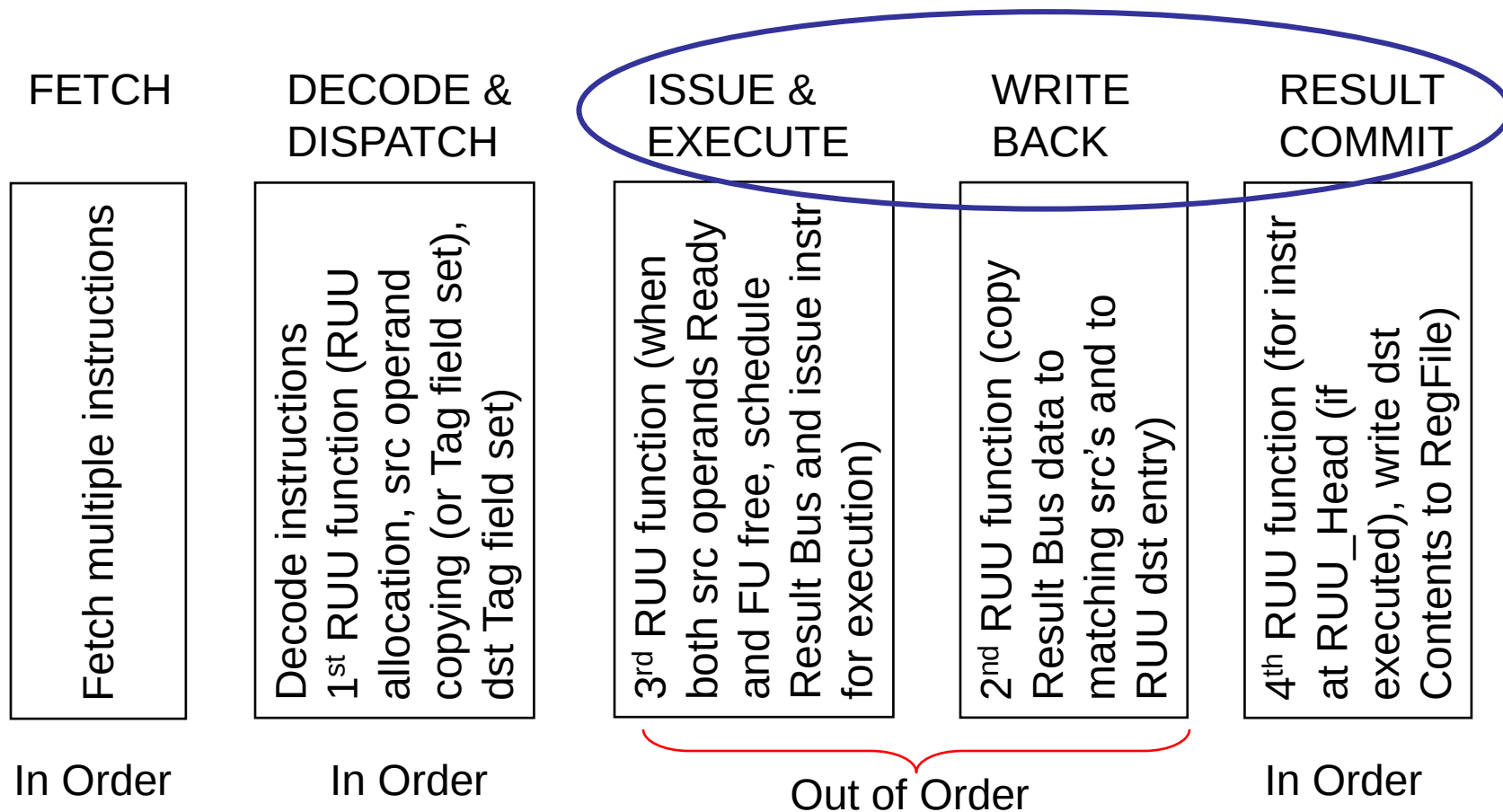
In Order

ISSUE & EXECUTE

3rd RUU function (when both
src operands Ready and FU
free, schedule Result Bus
and issue instr for execution)

Out of Order

SS Pipeline

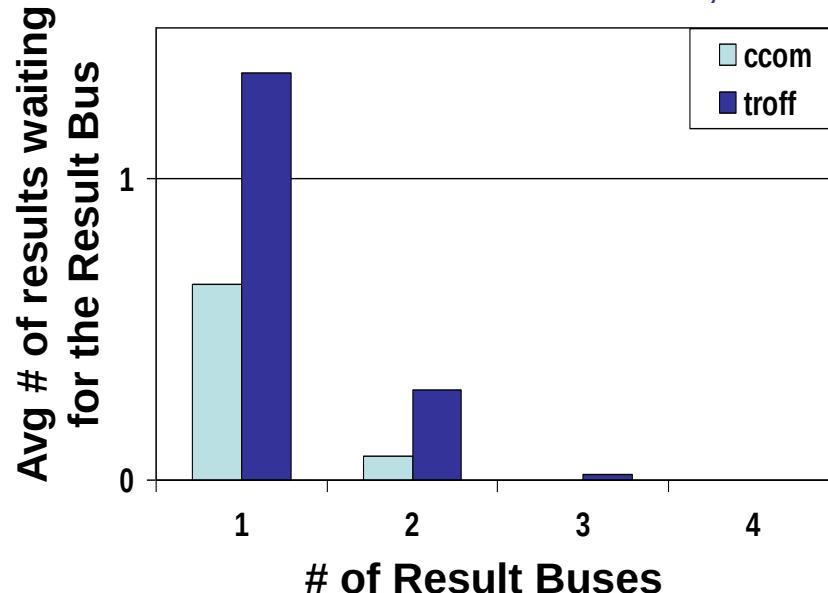


Result Buses Utilization

- Our SS model has only one Result Bus to carry results generated by the FU's to the RUU and LSQ
 - Even at the high levels of performance, the utilization of the Result Bus is only about 70% (i.e., the fraction of capacity actually used)

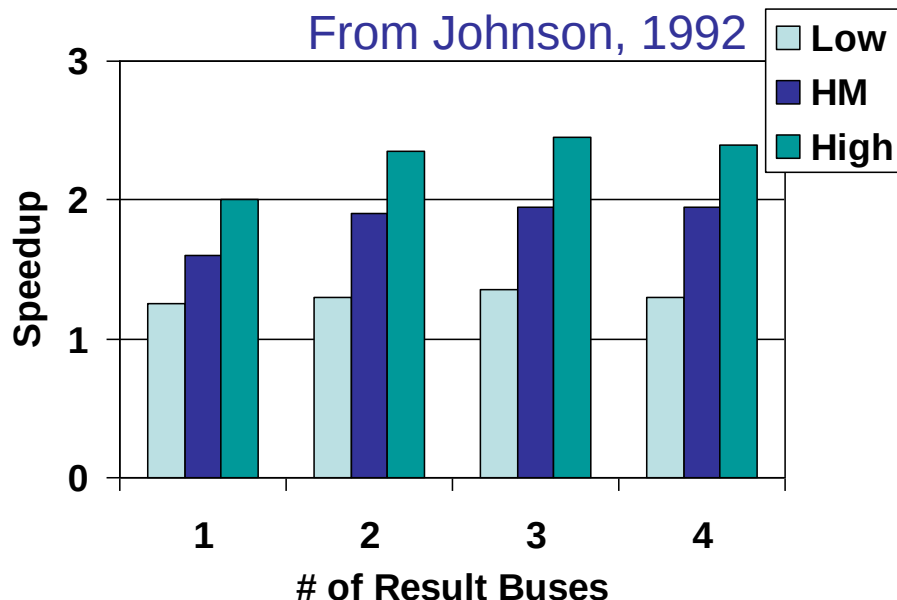
□ If a FU requests for the Result Bus is not granted, instruction issue to that FU is stalled until the bus request can be granted (i.e., FU remains “busy”)

From Johnson, 1992



Arbitrating For Result Buses

- ❑ Reducing the impact of bus contention by **adding a second Result Bus** improves performance (by almost 19%)
- ❑ But **adding a third Results Buses** yields only a very small improvement in performance (less than 3%)
- And since there are usually **fewer Result Buses than FUs**, the FUs must continue to arbitrate for use of the existing Result Buses
 - The arbiter not only decides which FU is granted use of the Result Buses, but also which of the two buses is to be used
 - Prioritizing old requests over new helps prevent starvation

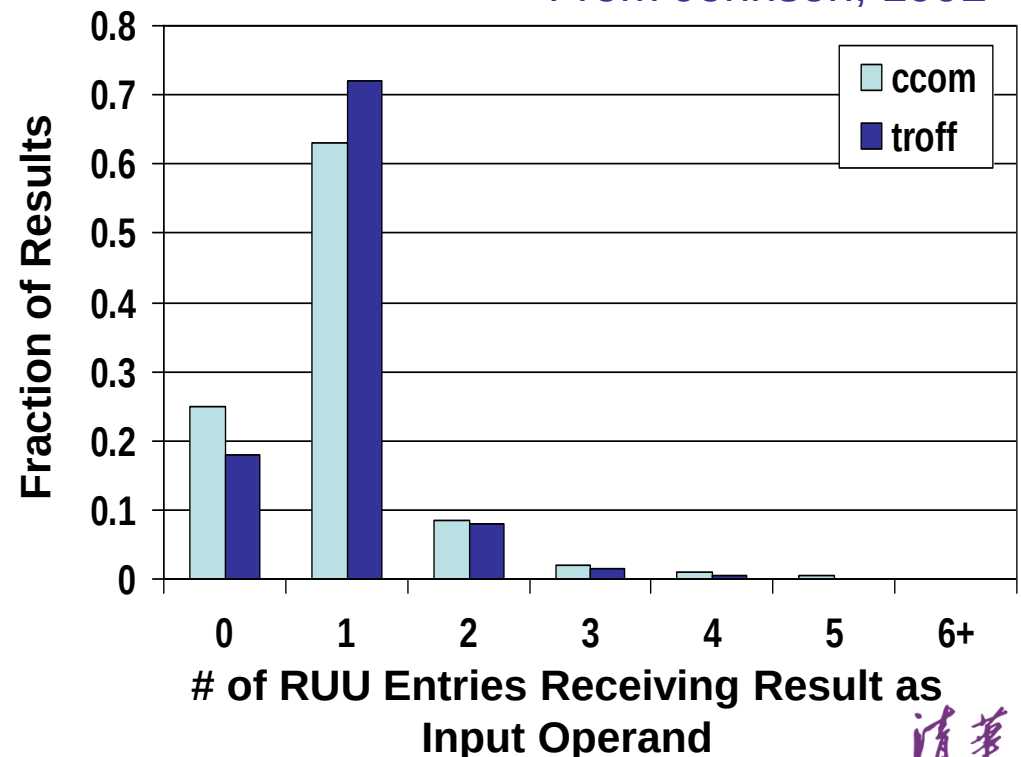


Result Forwarding

- **Result forwarding** supplies operands directly to the waiting instr's in the RUU to resolve true dependencies
 - The cost of forwarding is the comparison logic in the RUU to compare the Result Bus Tag to the source operand Tags
 - Need a set of comparators for each Result Bus

□ About $2/3^{\text{rd}}$ of all results are **forwarded** to one waiting operand, and about $1/6^{\text{th}}$ are **forwarded** to more than one

From Johnson, 1992



Performance Advantages

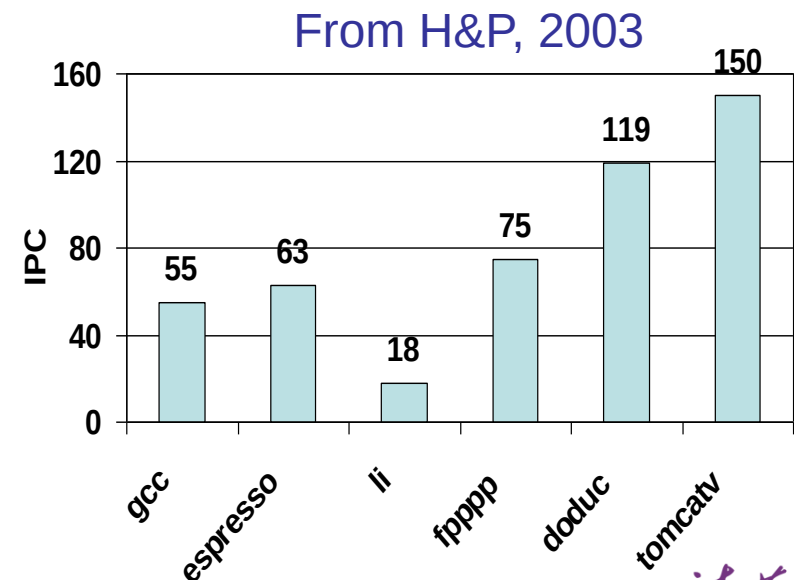
- Hardware complexity arises from four major hardware features
 - Out-of-order issue
 - Register renaming
 - Branch prediction
 - 4-way instruction fetch and decode

OOI	Register Renaming	Branch Prediction	4-way Fetch & Decode
52%	36%	30%	18%

The Performance Advantage of Major Processor Features,
From Johnson, 1992

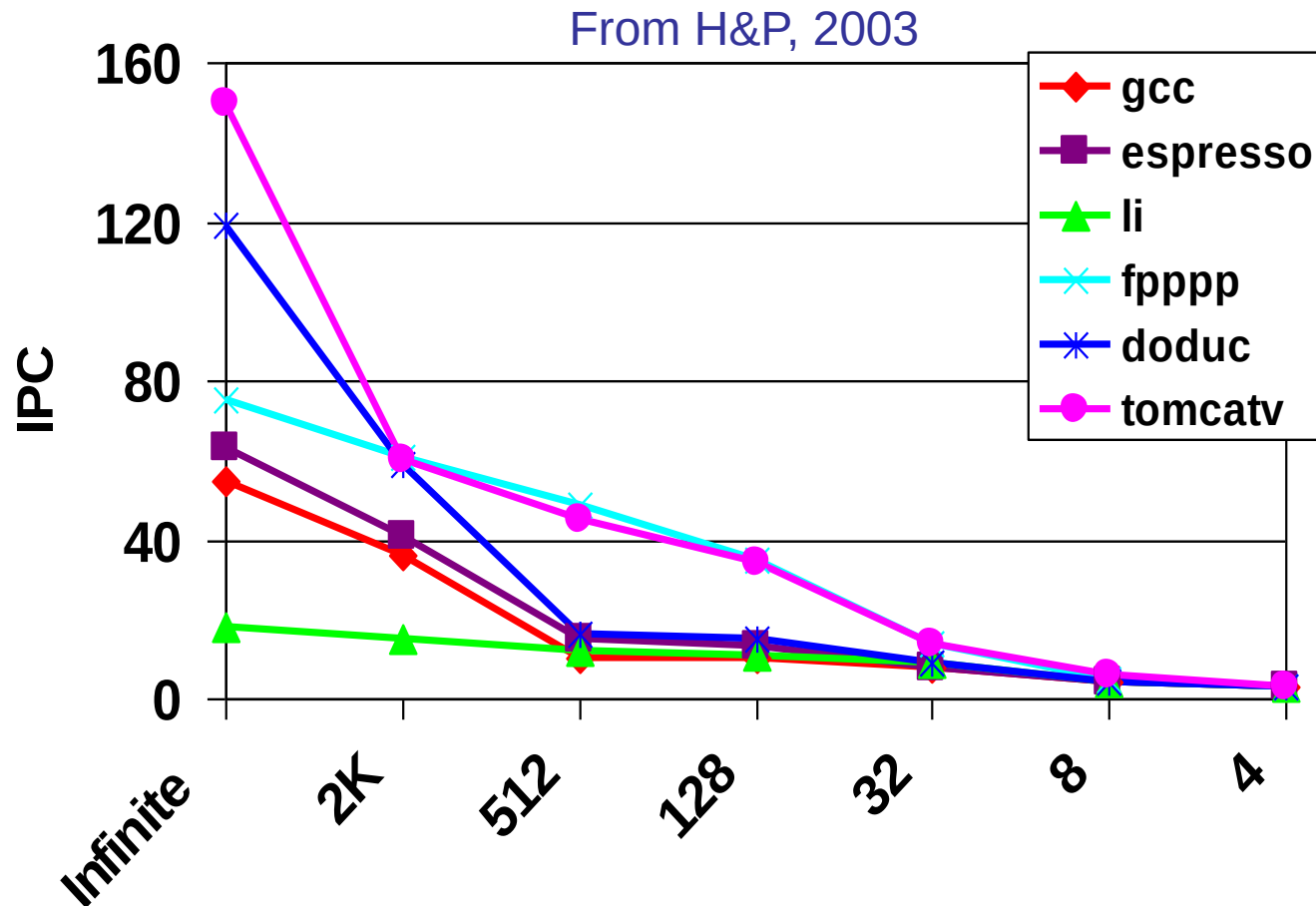
ILP in a Perfect OOI-OOC Processor

- The perfect processor has
 - An infinite number of rename registers that eliminates all storage hazards (i.e., write-before-write and write-before-read)
 - No (fetch, decode, dispatch, issue, FU, buses, ports) limit on the number of instr's that can begin execution simultaneously as long as read-before-write true data hazards are not present
 - Perfect branch and jump (including jump register) prediction
 - Perfect loads bypass
 - All FU's have a 1 cycle latency
 - Perfect caches with 1 cycle latency



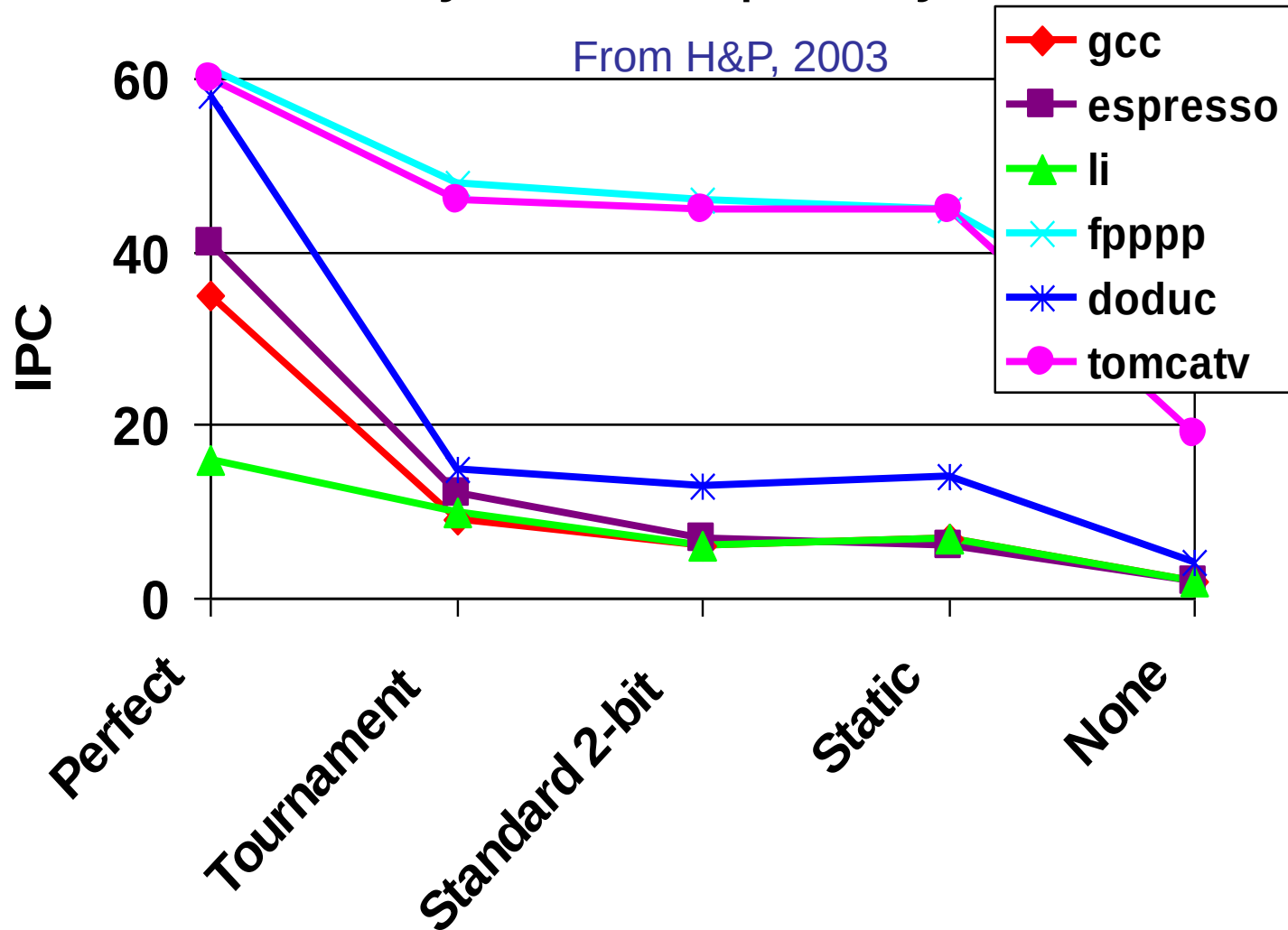
Effect of RUU Size on ILP

- RUU – the set of instructions that are examined simultaneously for execution



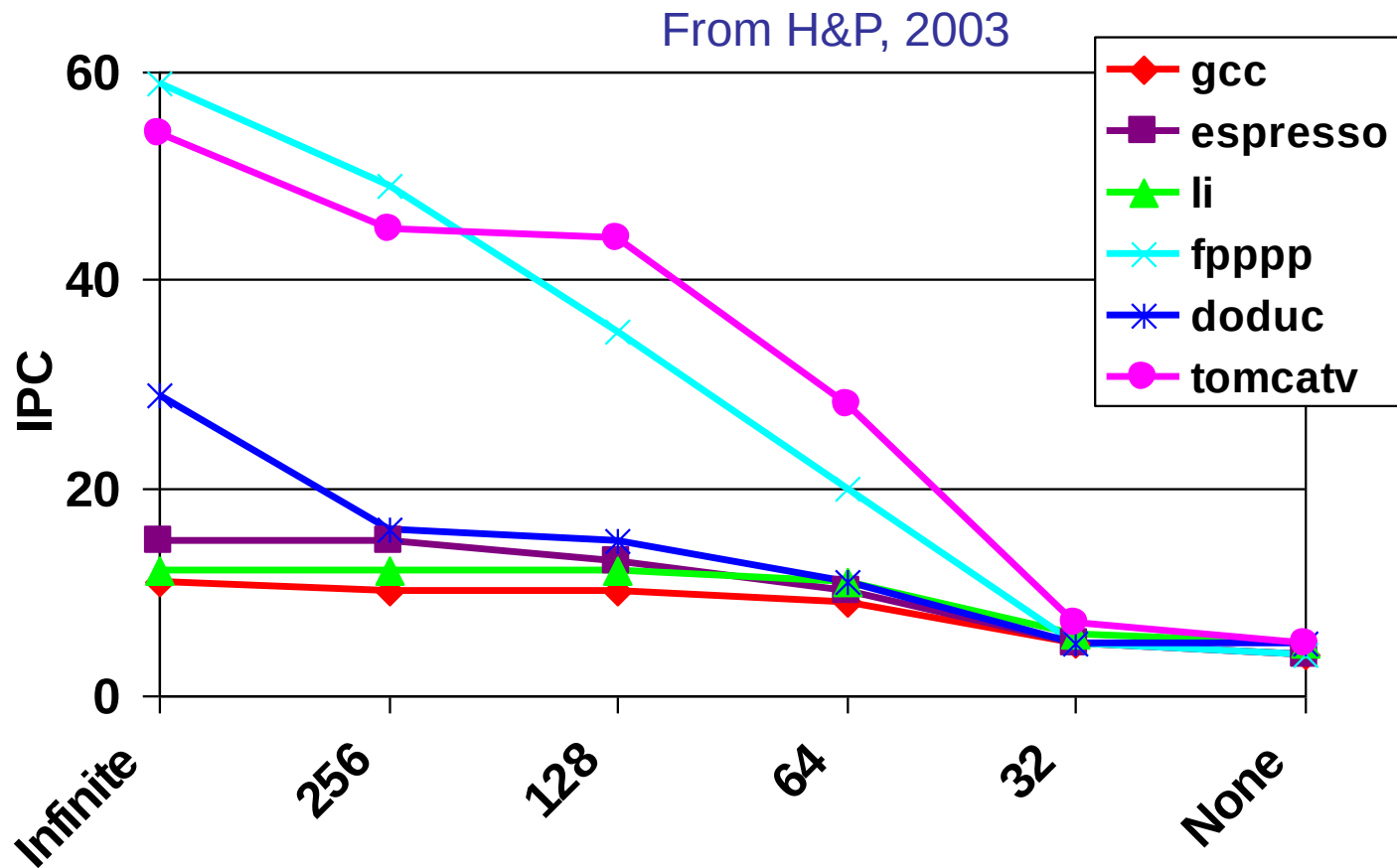
Effect of Realistic Branch Prediction on ILP

- On a processor with an RUU of 2K and maximum 64-way issue capability



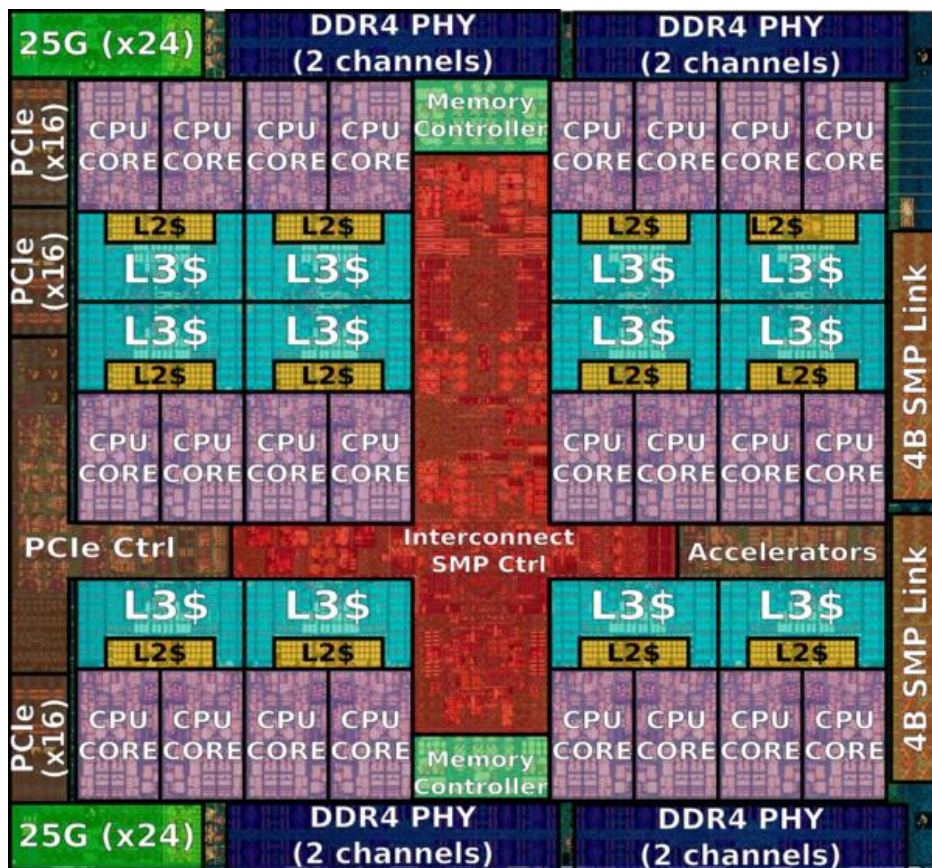
Effect of Finite Rename Registers

- On a processor with an RUU size of 2K, maximum 64-way issue capability, and a tournament branch predictor with 8K entries



A SS Example

- IBM Power9 Processor Architecture

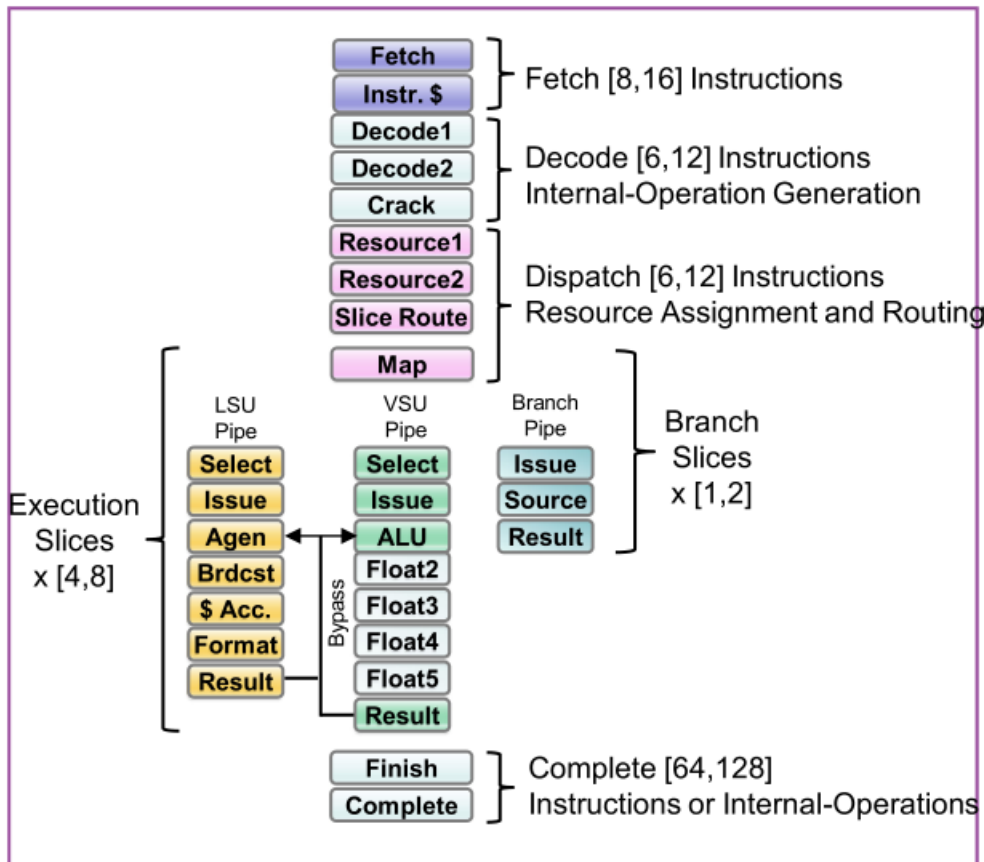


- 2017 年发布, 14 nm 工艺
- 最高支持 24 核心, 每核心 4 线程
- 设计用于:
 - Emerging analytics, AI, and cognitive computing;
 - Technical and high-performance computing (HPC);
 - Cloud and hyperscale datacenters;
 - Enterprise computing.

参考资料:

- Sadasivam, Satish Kumar, et al. "IBM Power9 processor architecture." *IEEE Micro* 37.2 (2017): 40-51.
- Le, Hung Q., et al. "IBM POWER9 processor core." *IBM Journal of Research and Development* 62.4/5 (2018): 2-1.
- <https://en.wikichip.org/wiki/ibm/microarchitectures/power9>

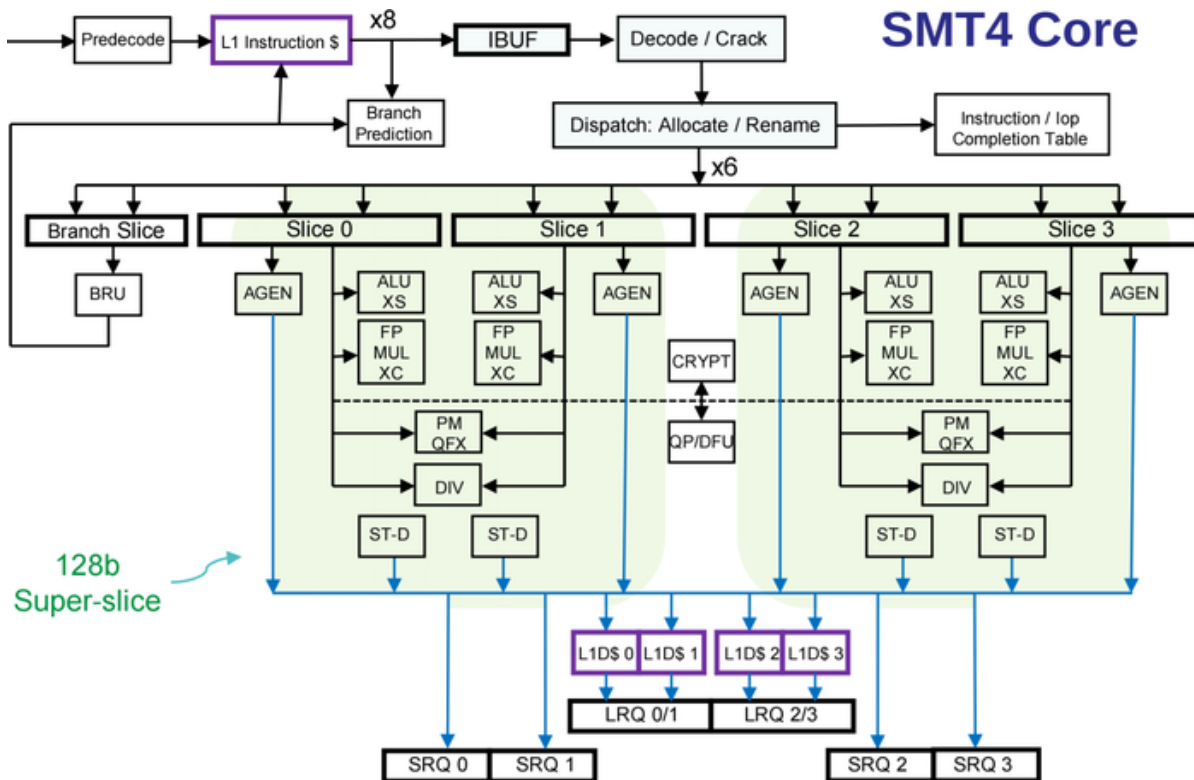
Power9 Core Pipeline



- Support completion of up to 256 instructions in one cycle.
- Advanced branch prediction including both direction prediction & target address prediction.
- Load store unit (LSU) to proactively avoid hazards.

*Values shown as [SMT4-core-rate, SMT8-core-rate]

Execution Slice Microarchitecture

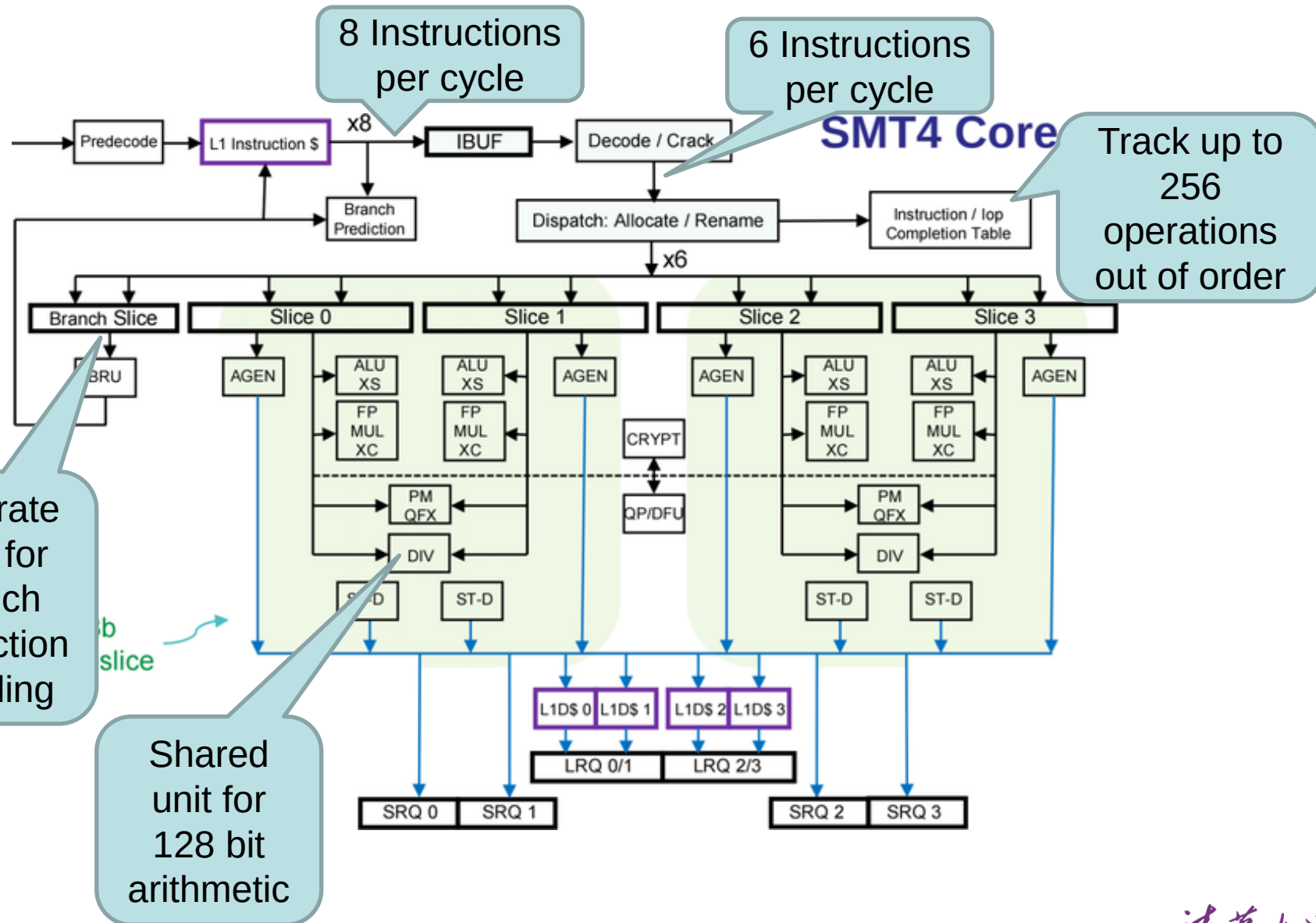


- A slice: 64-bit computational building block with a 64-bit load-store building block
- 2 slices = 1 super slice: handle 2 scalar or 1 vector operation
- A core contains 2 (SMT4) or 4 (SMT8) super slices

- 一个SMT4核可同时支持：
 - 4个简单定点操作 (ALU/XS)
 - 4个浮点和定点乘法，或其他复杂64 bit 定点操作 (FP/MUL/XC)

- 2个 128 bit permute operations (PM)
- 2个 128 bit quadword 定点操作 (QFX)
- 1个 128 bit quadword 浮点操作 (QP)
- 1个 decimal floating-point 操作 (DFU)
- 1个 crypto 操作 (CRYPT)

Execution Slice Microarchitecture



Branch Prediction

- Branch Prediction include:
 - Direction prediction [1]
 - A local selector to the Branch History Table (BHT) and a TAgged GEometric history length (TAGE) branch direction predictor [2]
 - Local selector tracks branches predicted well by the local predictor and prevents polluting global history vector.
 - Pattern-cache & branch target address calculator (BTAC) address predictor [3]

[1] Sinharoy, Balaram, et al. "IBM POWER8 processor core microarchitecture." *IBM Journal of Research and Development* 59.1 (2015): 2-1.

[2] Seznec, André, and Pierre Michaud. "A case for (partially) TAgged GEometric history length branch prediction." *The Journal of Instruction-Level Parallelism* 8 (2006): 23.

[3] Kalla, Ron, et al. "Power7: IBM's next-generation server processor." *IEEE micro* 30.2 (2010): 7-15.

Power ISA

- Broader Data-type
 - Native 128-bit floating point arithmetic
 - Instruction to convert between 16 bit and 32/64 bit FP data
 - Both integer and floating-point decimal arithmetic support
- Emerging Algorithms Support
 - Atomic memory operations for near memory computation: issued by processor but executed at memory controller
- Accelerator support
 - Virtual memory address mapping for on-chip & off-chip accelerators
- Secure computing
 - Single-instruction true random number generator
 - Arbitrary precision arithmetic

Summary

- SuperScalar SpeedUp Measurement
- FrontEnd :
 - Fetch and Decode
 - Branch Prediction
 - RUU/LSQ size
- BackEnd :
 - Result Bus
 - Result Forwarding
- Performance Evaluation